

华中科技大学

本科毕业设计（论文）参考文献译文本

译文出处：

H. Zhou et al. "Demystifying Diehard Android Apps." 2020 35th
IEEE/ACM International Conference on Automated Software
Engineering (ASE), 2020, 35: 187-198.

院 系	电子信息与通信学院
专业班级	种子 2101 班
姓 名	罗畅
学 号	U202113940
指导教师	袁巍

2025 年 2 月 17 日

译文要求

- 一、 译文内容须与课题（或专业内容）联系，并需在封面注明详细出处。
- 二、 出处格式为
图书：作者. 书名. 版本（第×版）. 译者. 出版地：出版者，出版年. 起页～止页
期刊：作者. 文章名称. 期刊名称，年号，卷号（期号）：起页～止页
- 三、 译文不少于 5000 汉字（或 2 万印刷符）。
- 四、 翻译内容用五号宋体字编辑，采用 A4 号纸双面打印，封面与封底采用浅蓝色封面纸（卡纸）打印。要求内容明确，语句通顺。
- 五、 译文及其相应参考文献一起装订，顺序依次为封面、译文、参考文献原文。
- 六、 翻译应在第七学期完成。

译文评阅

导师评语

应根据学校“译文要求”，对学生译文翻译的准确性、翻译数量以及译文的文字表述情况等做具体的评价后，再评分。

评分：_____（百分制）

指导教师(签名)：_____

年 月 日

揭秘顽固安卓应用程序

摘要

为了减少电池消耗，智能手机制造商正在使用各种方法杀死安卓应用程序的进程。这促使开发者们寻找新的延长他们的应用程序存活时间的手段，因此，本文将他们称为顽固应用（Diehard APPs）。即使现在已经有很多博客和文章详细阐述了实现手段，但仍然没有关于他们的系统性研究。更重要的是，我们对这种顽固应用程序的在野流行度知之甚少。

在本文中，我们迈出了第一步，通过回答下面的问题来系统性地对顽固应用程序做出了研究。第一，为什么以及如何规避 Android 系统节省资源的机制？第二，如何对这种现象的在野流行度进行评估？除此之外，我们进行了半自动化的分析，以说明为什么可以逃避现有的杀死应用程序进程的方法，然后系统地提出了 12 种保活顽固方法。之后，我们开发了一个名为顽固检测器（DiehardDetector）的系统来大规模检测顽固应用。将顽固检测器应用于从 Google Play 下载的超过 8 万个安卓应用的实验结果表明，约 21% 的应用采用了各种顽固方法。此外，我们的系统可以达到较高的准确率和召回率。

1 简介

电池续航是手机用户的主要关注点之一。一个近期的调查显示，41% 的美国智能手机用户表示，更长的电池续航是他们最希望拥有的设计特性^[9]。因此，智能手机制造商部署了很多节省资源的机制来杀死后台应用程序以优化电池使用^[3]。

然而，从另一个方面来说，软件开发者倾向于提升他们的软件应用存活时间。这样做他们可以与用户保持非活跃的交互体验，比如提供及时服务（例如发送通知），创造收益（例如推送广告）等。可以肯定的是，近年来，已经有很多应用程序采用了各种方式来绕过节省资源的机制，这些方法可以延长顽固应用的寿命，称为“顽固的手段”

可以根据目的，将顽固手段方法大致分为两类：（1）使应用程序保持持续存活；（2）唤醒应用程序。Shao 等人^[26]手动研究了 23 个顽固应用程序并找到了一些顽固手段，其中他们的大多数都旨在唤醒应用程序。但是，学术界仍然对顽固的手段和顽固应用程序缺少深入的了解。

在本文中，我们迈出了第一步，通过回答三个研究问题，系统性地研究了顽固软件应用程序：(1) 为什么现有的杀死软件进程的方法能被绕过？(2) 这些顽固应用程序是如何让自己保持存活或被唤醒的？(3) 顽固行为是否广泛存在于在野应用？回答这些问题，可以促进开发高效的资源节省调度机制以使用户受益（即一个电池利用高效的系统），并通过发现顽固应用程序促进软件应用生态健康发展。

为此，我们首先对现有杀死应用程序进程的方法进行了系统研究，并分析出为什么可以绕过这些方法（第 3 节）。找出这些方法并找出相应的绕过手段并不容易，因为安卓系统有超过 100 万行代码，并且在不断发展，更不用说这些方法缺乏详细的文档。我们使用半自动化的方法来应对这一挑战，用静态代码分析提取大多数相关的源代码，然后手动检查它们以了解其内部机制。

第二，利用上一步中获得的观察结果，我们系统地提出了 12 种顽固手段，包含 7 种可以使其保持存活的手段（第 4 节），它们中的 6 种此前都没有被报道过；剩余的 5 种手段则可以唤醒顽固应用（第 5 节），它们中的 1 种在先前的工作中没有被提及到^[26]。为了评估这些方法的有效性，我们开发了两个顽固应用程序（一个普通的应用和一个及时应用程序（即快应用，下同））。从安卓 5.1 到安卓 10 的流行安卓版本的广泛实验表明，我们所有的方法都可以延长应用程序的存活时间。

第三，为了检测在野的顽固应用程序，我们开发了顽固检测器（DiehardDetector），这是一种基于 12 个顽固方法的新检测工具。我们认真设计了顽固检测器，使其得到最低的假阳性和假阴性率，并且能够检查大量的安卓应用。顽固检测器的评估结果表明它是准确（检测到顽固应用的结果精确度和召回率分别是 100% 和 94.70%）和有效的（平均分析一个应用程序需要 45 秒）。

第四，我们用顽固检测器分析了超过 8 万个应用以识别顽固手段的使用。实验结果表明，大约 21% 来自 Google Play 的应用程序有顽固行为。此外，在进行实验的应用程序中，有两种被我们提出的顽固手段在这些应用程序中尚未被发现，这表明我们发现了两种以前未知的延长应用存活时间的顽固手段。

为了吸引社区的关注，我们将顽固检测器和涉及实验的应用程序在 [github](#) 上进行了开源。

总的来说，本文做出了如下贡献：

- 据我们所知，我们是第一个对顽固应用程序进行系统性研究的。特别的是，我们是第一

个揭示为什么可以规避资源节约调度机制的团队，并提出了 12 种保持应用程序存活和唤醒应用程序的顽固手段；

- 我们构建了顽固检测器——一个可以准确有效识别安卓应用中顽固行为的新工具；

我们进行了广泛的实验，以评估顽固检测器的有效性并评估顽固方法的在野流行度。

在 8 万多个来自 Google Play 的应用程序上进行的实验表明，顽固检测器能准确地、以可扩展的方式检测出应用存在的顽固行为。此外，大约有 21% 在 Google Play 上发布的应用程序具有顽固行为。

2 背景

安卓应用可以被大致分为两类：(1) 手机厂商预装的系统应用，因为安卓系统通常会保护这些应用不被杀掉，所以这些应用很少使用顽固手段；(2) 普通应用程序和快应用，这些应用由开发人员开发并被智能手机用户下载使用。这些应用通常会使用保活手段来延长其存活时间，以达到多种目的（例如推送广告以获取利润）。

一个应用程序可能会有多个进程。默认情况下，应用程序的组件（即活动——Activity、服务——Service、广播接收器——Broadcast Receiver 和内容提供者——Content Provider）在程序的主进程运行。但是，每个组件都可以在清单文件（manifest）中声明其进程属性，以使其在主进程以外的其他进程中运行。如果应用程序所有的运行进程都被杀死，则该应用程序将被终止。

为了管理应用程序进程，安卓系统框架提供了两种数据结构：ProcessRecord 和 TaskRecord。我们利用它们来找出杀死或停止应用程序的方法。请注意，停止应用程序的方法是杀死应用程序进程的特殊情况，因为它们旨在杀死与应用程序相关的所有运行进程。

ProcessRecord: 它被安卓系统中最核心的服务之一——ActivityManagerService (AMS) 用来表征每个运行的应用程序进程。具体来说，一个 ProcessRecord 对象的 curAdj 字段和 curSchedGroup 字段（或 setSchedGroup）字段表征一个应用程序进程的优先级值。有多种方法可以通知 AMS 为一个应用程序创建一个 ProcessRecord 对象，例如启动它的 Activity、启动它的 Service，发送一个可以被它的 Broadcast Receiver 接受的广播，访问它的 Content Provider 等。

TaskRecord: 被用于表征一个 Activity 任务^[11], Activity 任务是用户在执行某个任务时阈值互动的 Activity 的集合。同时, 在正常情况下, 最近访问的应用程序的 TaskRecord 对象对应于最近任务列表中的一个项目^[9]。因此, TaskRecord 对象将最近访问的应用程序的 Activity 记录与最近任务项目连接起来。

3 杀死应用程序进程的方法

为了找到杀死应用程序进程的方法, 我们对安卓 9.0 的框架和系统应用程序 (例如低内存杀手, Low Memory Killer, LMK) 进行了半自动调查。我们从这些调查结果中了解到, 应用程序进程的优先级会影响它是否将被杀死, 并在 3.2 小节中阐释安卓系统是如何确定应用程序进程的优先级的。在 3.3 小节中, 我们揭露了这些杀死进程方法的内部机制, 并指出了为什么可以被规避, 然后将其利用为设计顽固的手段 (在第 4 和第 5 节中)

3.1 寻找杀死进程的方法

顽固应用程序使用顽固手段来防止其进程被杀死。因此, 我们寻找所有杀害进程的方法, 以了解为什么它们可以被规避。有两种基本方法可以终止应用程序进程: (1) 调用 ProcessRecord 类中定义的 Kill API 接口, 因为安卓框架使用 ProcessRecord 来表示运行的应用程序进程; (2) 使用 lib.so 中的 kill 函数, 因为原生应用程序或库可以调用该函数, 而该函数会进一步调用系统调用 __NR_kill。我们采用一种半自动的方法来找到其他杀死进程的方法, 这些方法最终将调用上述基本方法中的其中一个。从基本方法喀什, 我们使用静态分析来提取最相关的源代码, 然后如下段落所述手动检查它们。值得注意的是, 这种半自动的方法使我们可以检查整个安卓系统, 并且可以在未来的安卓版本中用于寻找新的或更改的杀进程方法。

过程: 要在安卓系统养家中找到杀死进程的方法, 我们首先使用 PScout 构建其调用图^[13]。然后, 我们在调用图中找到 ProcessRecord.kill API, 并执行向后的可及性分析仪获取到达此 API 的执行路径, 每条执行路径都设计一种特定的杀进程方法。此外, 要揭示终止应用程序进程的条件, 我们对每个执行路径执行控制流分析以找到影响 kill API 执行的条件语句。随后, 我们首都奥那个分析提取的条件成熟, 以披露如何规避杀进程的方法。

例如, 图 1 显示了杀死所有后台进程的方法的执行路径和必要的条件语句 (3.3 小节中的 KAP)。我们找到了一条可以访问 ProcessRecord.kill API 的执行路径 M1→M2→M3→M4

→M5（即 M5）。然后，我们找到了达到 M5 需要满足的条件（即 C1 和 C2）。在检查了这些条件语句（特别是 C2）后，应用程序进程的优先级（即在相应的 ProcessRecord 实例的 setAdj 中储存的值）确定了该应用程序是否将被杀死。如果应用程序的优先级不满足 C2，则此应用程序将绕过 KAP。

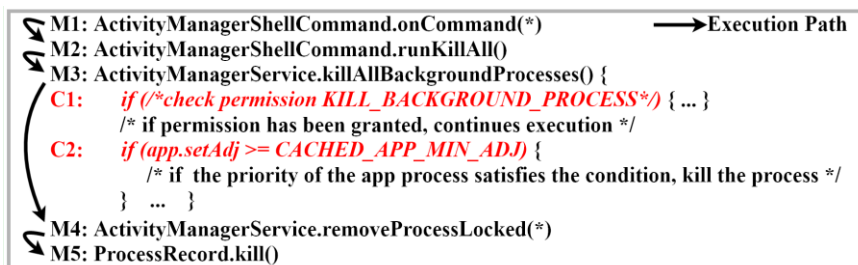


图 1 找到进程杀死方法，KAP

因为安卓系统的原生软件可以调用 lib.so 中的 kill 函数，为了寻找其中的杀进程方法，我们使用 SVF^[27]为每个原生应用程序构建了调用图，并定位该 kill 函数。一旦找到，我们从 kill 函数开始反向遍历调用图，以获取到达该函数的执行路径。然后，我们手动分析每条执行路径中方法的源代码，研究如何规避进程终止方法。

结果：我们找到了 6 个杀死进程的方法（在 3.3 小节中），他们中的 4 个检查了应用程序进程的优先级，以决定是否将其杀死。同时，我们注意到，在杀死应用程序后，安卓框架将调用 AMS 的 updateOomAdjLocked 函数来调整当前运行应用程序进程的优先级。为了了解 AMS 如何计算应用程序进程的优先级，我们对 AMS 的 computeOomAdjLocked 函数执行控制流分析，该函数通过 updateOomAdjLocked 函数调用。具体而言，由于局部变量 adj 和 schedGroup 分别被用于更新 ProcessRecord 实例的 curAdj 和 curSchedGroup 字段，因此我们提取了决定是否更改其值的条件语句。然后，我们手动分析了这些发现，以揭露为什么一个进程具有特定优先级（在 3.2 节中）

3.2 应用程序进程优先级

安卓系统使用内存溢出（OOM）调整和线程调度组确定应用程序进程的优先级。在下文中，我首先介绍如何设置应用程序进程的 OOM 调整值，该过程与 4.2 小节中提出的顽固手段相关，然后描述与 4.1 小节中提出的顽固手段相关的两个线程调度组。表 1 总结了应用程序进程的差异案例。

OOM 调整值。优先级高的应用程序进程通常具有较小的 OOM 调整值，并且被杀死的可能性也较小。

- **FOREGROUND_APP_ADJ**: 前台应用程序的进程（例如 C01 和 C02）与此 OOM 调整值有关。
- **VISIBLE_APP_ADJ**: 持有可见 activity 的应用程序进程（例如 C05）与此 OOM 调整值有关。
- **PERCEPTIBLE_APP_ADJ**: 持有可感知组件（例如与可见通知连接的前台 Service）的应用程序进程（例如 C08）与此 OOM 调整值有关。
- **SERVICE_ADJ**: 持有正在运行的 Service 的应用程序进程（例如 C12）的优先级由此 OOM 调整值表示。
- **CACHED_APP_MIN_ADJ**: 持有不可见 Activity 的应用程序进程（例如 C13）与此 OOM 调整值有关。

注：如果一个应用程序进程满足以上的多个描述，则他的 OOM 调整值取最小值。例如，一个持有可见 Activity 和正在运行的 Service 的应用程序进程的 OOM 调整值为 **VISIBLE_APP_ADJ**。

线程调度组。AMS 用它来管理一组应用程序进程的活跃度。通常情况下，具有相同优先级的进程往往被组织在一个线程调度组中。

- **SCHED_GROUP_TOP_APP**: 该组包含优先级较高的前台应用程序（例如 C01）。
- **SCHED_GROUP_BACKGROUND**: 该组包含优先级相对较低的后台应用程序进程（例如 C12）。

表 1 不同应用程序进程情况汇总

OOM 调整值	值	案例	调度组
FOREGROUND_APP_ADJ	0	C01: 进程具有前台 Activity，并且设备处于唤醒状态。	SCHED_GROUP_TOP_APP (3)
		C02: 进程具有前台 Activity，但设备处于睡眠状态。	SCHED_GROUP_BACKGROUND (0)
		C03: 进程具有 Service，另一个具有此类 OOM 调整的进程绑定到该 Service。	SCHED_GROUP_DEFAULT (2)

		C04: 进程具有提供者, 另一个具有此类 OOM 调整的进程获取了该提供者。	SCHED GROUP_DEFA ULT (2)
VISIBLE_APP_ ADJ	100 ~ 200	C05: 进程具有可见 Activity。	SCHED GROUP_DEFA ULT (2)
		C06: 进程具有 Service, 另一个具有此类 OOM 调整的进程绑定到该 Service。	SCHED GROUP_DEFA ULT (2)
		C07: 进程具有提供者, 另一个具有此类 OOM 调整的进程获取了该提供者。	SCHED GROUP_DEFA ULT (2)
PERCEPTIBLE _APP_ADJ	200	C08: 进程具有前台 Service。	SCHED GROUP_DEFA ULT (2)
		C09: 进程具有覆盖窗口。	SCHED GROUP_DEFA ULT (2)
		C10: 进程具有 Service, 另一个具有此类 OOM 调整的进程绑定到该 Service。	SCHED GROUP_DEFA ULT (2)
		C11: 进程具有提供者, 另一个具有此类 OOM 调整的进程获取了该提供者。	SCHED GROUP_DEFA ULT (2)
SERVICE_ADJ	500	C12: 进程具有 Service, 该 Service 在过去 30 分钟内启动。	SCHED GROUP_BACK GROUND (0)
CACHED_APP _MIN_ADJ	900	C13: 进程具有已停止的 Activity。	SCHED GROUP_BACK

			GROUND (0)
--	--	--	------------

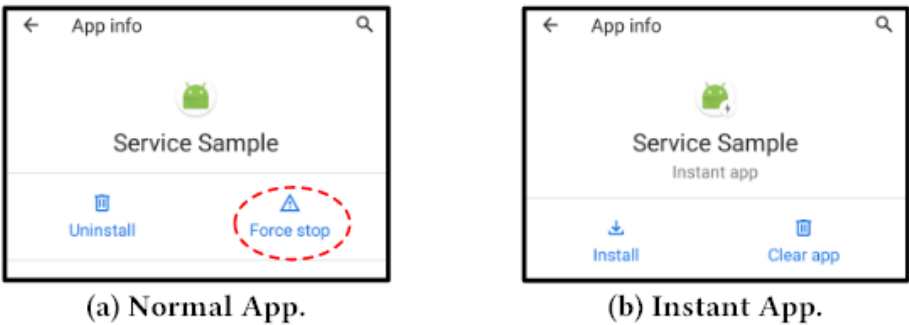


图 2 不同类型的程序的应用信息界面，(a) 普通应用，(b) 快应用

3.3 探寻杀死进程的手段

在本节中，我们详细阐述了 3.1 小节中发现的 6 种杀死进程方法的内部机制，特别是决定一个应用进程是否会被杀死的条件，并指出了这些方法被规避的原因。

1. 移除最近任务项（Remove Recent-Task Item, RRT）

使用：用户可以在“最近任务”列表中滑动刷去显示的项目，以停止最近访问的应用程序。

内部机制：RRT 由两个主要步骤组成，首先，AMS 从 ActivityStackSupervisor 的 mRecentTasks 字段中移除与移除的“最近任务”项相关的 TaskRecord 对象，该阻断是一个记录每个最近访问的应用的 TaskRecord 对象的数组。然后，AMS 使用 ActivityStackSupervisor（ASS）即 Activity 栈管理器来获取与带停止应用相关的 ProcessRecord 对象。一旦找到 ProcessRecord 对象，ASS 将检查该进程是否持有 Activity。如果没有，这个进程进程就是被杀死的候选进程。否则，ASS 检查进程通过的 Activity 是否属于同一活动任务，如果是，这样的进程也是候选进程。在调用 ProcessRecord 的 kill 方法中值候选进程之前，ASS 检查进程的线程调度组是否为 SCHED_GROUP_BACKGROUND，如果是，则该进程将被杀死。

注：RRT 不能阻止没有最近任务项的应用程序。而且，它既不会杀死具有多个 Activity 任务的应用进程，也不会杀死线程调度组不是 SCHED_GROUP_BACKGROUND 的应用程序进程。

2. 强制停止应用程序（Force-stopping APP, FSA）

使用：用户可以导航到应用程序信息界面，然后单击“强制停止”按钮以停止应用程序。调试器可以使用 shell 命令 am force-stop 来实现相同的目的。

内部机制：FSA 需要 3 个步骤来停止应用程序。首先，AMS 通知 PackageManagerService（PMS）的实例，以使目标应用程序忽略接收到的未携带标志 FLAG_INCLUDE_STOPPED_PACKAGES 的广播。然后，AMS 找到与应用程序关联的 ProcessRecord 对象，并调用杀死方法以终止流程。第三，AM 发送广播以指示 AlarmManagerService 和 JobSchedulerService 的实例清理该应用程序先前提交的已待处理任务。同时，广播将由 NotificationManagerService 收到，与目标应用程序相关的通知将相应驳回。

注：在正常情况下，FSA 可以杀死应用程序的所有正在运行的进程。但是，我们发现快应用是一个例外。如图 2b 所示，快应用的应用程序信息界面没有强制停止按钮。也就是说，FSA 无法停止快应用，而活着的快应用可以唤醒其他应用程序。

3. 杀死后台进程（Killing Background Processes, KBP）

使用：具有权限 KILL_BACKGROUND_PROCESSES 的应用程序可以终止后台应用程序。更具体地说，该应用首先获得 ActivityManager 的实例，然后调用 ActivityManager 的 killBackgroundProcesses 方法杀死后台应用程序。调试器可以执行 Shell 命令 am Kill 以完成相同的任务。

内部机制：KBP 的工作流程包含 2 个步骤。首先，AMS 检索所有存活的 ProcessRecord 对象，并检查每个对象的 setadj 字段，以确定是否将其作为待杀进程。确切地说，如果候选进程的 OOM 调整值不小于 SERVICE_ADJ，则将被停止。其次，AMS 调用 ProcessRecord 中的 kill API 以杀死目标。

注：KBP 的第一步意味着它不能杀死 OOM 调整值小于 SERVICE_ADJ 的进程。

4. 杀死所有后台进程（Killing All Background Processes, KAP）

使用：调试器可以使用 shell 命令 am kill-all 杀死在后台运行的所有应用程序进程，因为此命令使 AMS 调用其 killAllBackgroundProcess 方法。

内部机制：KAP 的工作流与 KBP 相似，其差异在于找到目标应用程序进程的标准。确切地说，KAP 杀死了 OOM 调整值不小于 CACHED_APP_MIN_ADJ 的应用程序进程。

注：OOM 调整值小于 CACHED_APP_MIN_ADJ 的应用程序进程将被排除在 KAP 的目标之外，因此不会被 KAP 杀死。

5. 杀死特定进程（Killing Specific Process, KSP）

使用：应用程序或调试器可以分别使用原生的 kill 函数和 kill 命令根据 PID 杀死特定进

程。请注意，应用程序只可以杀死自己的流程，而特权调试器没有这种约束。

内部机制：KSP 调用原生杀死功能以终止目标。

注：KSP 无法阻止应用程序被唤醒。

6. 低内存杀手（Low Memory Killer, LMK）

使用：如果运行设备的可用 RAM 变得不足，LMK 的本机守护程序将杀死一些选定的应用程序进程。

内部机制：LMK 包括 2 个主要步骤。首先，根据剩余的可用 RAM 值决定 OOM 调整值的阈值。其次，列出 OOM 调整值超过阈值的应用程序进程。在这些过程中，LMK 使用原生 kill 函数来杀死占据最多 RAM 的应用进程。然后，LMK 反复执行这两个步骤，直到没有进程的 OOM 调整值超过阈值。

注：第二步表明，LMK 很少杀死高优先级的应用程序进程（即较小的 OOM 调整值）。

4 保持应用程序活跃

在本节中，我们通过利用第 3.3 小节中得出的结论，我们提出了如表 2 所示的 7 种顽固的手段，以使应用程序保持活跃。请注意，如果应用程序还有在系统中运行的进程，则表明该应用程序还活着。

表 2：保持应用程序活跃的顽固手段

方法	是否支持 快应用	杀死应用进程的方法					
		RRT	FSA	KBP	KAP	KSP	LMK
HTI	√	●	●	⊗	⊗	⊗	⊗
PMI	√	●	●	⊗	⊗	⊗	⊗
HFA	√	⊗	⊗	●	●	⊗	●
HFS	√	●	●	●	●	⊗	●
COW	√	●	●	●	●	⊗	●
BRS	√	●	●	●	●	⊗	●
ACP	√	●	●	●	●	⊗	●

注：●表示该方法可以防止普通应用和快应用被杀死；●表示该方法可以防止普通应用或快应用被杀死；⊗表示该方法无法阻止普通应用和快应用被杀死。

4.1 操纵最近任务项

- **隐藏最近任务项（Hiding Recent-Task Item, HTI）**

设计：由于 RRT 只会停止最近任务列表中包含任务项的应用，因此该应用可以隐藏自己的任务项以逃避 RRT。

实现：为了隐藏最近任务项，应用程序可以为在其 manifest 文件或代码中将其主要可启动 Activity 的 `excludeFromRecents` 属性设置为真。对于后者，应用首先获取 `ActivityManager` 实例，调用 `ActivityManager` 的 `getAppTasks` 方法，得到与最近任务项绑定的 `AppTask` 对象。然后，应用调用 `AppTask` 类中定义的 `setExcludeFromRecents` 方法来移除最近任务项。

- **产生多个最近任务项（Producing Multiple Recent-Task Items, PMI）**

设计：由于 RRT 杀死的是在 Activity Task 中组织托管 Activity 的进程，为了规避它，应用程序可以创建多个 Activity Task，从而产生多个最近任务项。

实现：为了使安卓创建一个额外的 Activity Task 用于存储该 Activity，用于启动目标 Activity 的 Intent 对象应携带标志 `FLAG_ACTIVITY_NEW_TASK`^[23]。注意，如果源 Activity(即 intent 发送者)和目标 Activity(即,intent 接受者)的 `taskAffinity` 属性相同,则 Android 不会创建新的 Task。为了防止这种情况，应用程序应该让目标 Activity 的 `taskAffinity` 属性与源 Activity 的 `taskAffinity` 属性不同。

4.2 提升应用程序优先级

- **持有前台 Activity（Holding Foreground Activity, HFA）**

设计：如表 1 所示，持有前台 Activity 的应用程序进程(即 C01 或 C02)具有最小的 OOM 调整值。这样的进程不会被 KBP 和 KAP 杀死，因为他们选择优先级较低的应用程序进程。

实现：为了让后台程序持有一个前台 Activity，应用程序可以声明或注册一个收听系统/应用程序广播的接收器。一旦应用程序收到广播，它就会在前台启动一个 Activity。顽固应用程序可以进一步使 HFA 难以被注意。例如，应用程序监听系统广播 `ACTION_SCREEN_OFF`，并在设备屏幕关闭时启动前台 Activity，或启动非常小的 Activity。

- **持有前台 Service（Hosting Foreground Service, HFS）**

设计：持有前台 Service 的应用程序进程（例如 C08）的 OOM 调整值为 `PERCEPTIBLE_APP_ADJ`，小于 `SERVICE_ADJ` 和 `CACHED_APP_MIN_ADJ`，因此这类应用程序进程不会被 KBP 或 KAP 杀死。同时，由于相关线程调度组是 `SCHED_GROUP_DEFAULT`，RRT 无法杀死此类应用程序进程。

实现：应用程序首先通过创建 Intent 对象并调用 startService 或 bindService 发起 Service。然后，为了使 Service 前台化，在 Service 的 onCreate、onStartCommand 或 onBind 回调中，应用程序创建一个 Notification 对象，并将其传递给 startForeground API，该 API 会显示通知，以通知用户存在前台 Service。

- **创建叠加窗口（Creating Overlay Window, COW）**

设计：一个应用程序进程（例如 C09）可以启动一个覆盖窗口^[17,22]，该窗口将被渲染在任何其他的应用窗口（例如应用程序 Activity 的界面）之上，将其现成调度组调整为 SCHED_GROUP_DEFAULT，并将其 OOM 调整值提升为 PERCEPTIBLE_APP_ADJ。所以，这样的应用程序进程将不会被 RRT, KBP, KAP 杀死。

实现：在安卓 8.0 之前，为了使应用程序窗口成为覆盖窗口，应用程序调用 Window 类中定义的 setAttribute 或 setType 方法将窗口类型设置为 TYPE_PHONE、TYPE_SYSTEM_ALERT、TYPE_SYSTEM_ERROR 或 TYPE_TOAST。对于发布的更新版本的安卓系统（即发布不早于安卓 8.0 者），通常采用 TYPE_APPLICATION_OVERLAY 来构造覆盖窗口^[32]，因为前述窗口类型已经被弃用。

- **绑定正在运行的 Service（Binding Running Service, BRS）**

设计：一个应用可以在单独的进程（例如 Service 进程）中运行其 Service，以减少其主进程的资源占用。由于 Service 进程通常不会托管/持有任何用户可见的组件（例如 UI），因此其 OOM 调整值大于 PERCEPTIBLE_APP_ADJ。因此，为了防止 Service 进程被杀死，应用程序需要提升其 Service 进程的优先级。在仔细检查 AMS 如何计算和恭喜应用程序进程的优先级后，我们发现 Service 进程的优先级会受到其绑定到 Service 的客户端进程（例如 C03, C06 和 C10）的优先级的影响。具体来说，如果 Service 进程的 OOM 调整值大于其部分客户端进程的 OOM 调整值，则将其调整到所有客户端基础中最小的一个，并将现成调度组设置为 SCHED_GROUP_DEFAULT，这样 Service 进程就不会被 RRT, KBP 和 KAP 杀死。

实现：应用程序进程调用 bindService 方法将自身绑定到一个运行在独立进程中的 Service，值得注意的是，一旦所有的客户端进程通过调用 unbindService 方法将自身与 Service 断开连接，该 Service 进程的优先级将不会被客户端进程影响。

- **获取已发布的 Content Provider（Acquiring Published Content Provider, ACP）**

设计：Content Provider 可以在单独的应用程序进程（例如 Provider 进程）中运行，其优先级会受到其客户端进程（即 C04、C07 和 C11）的影响。Provider 进程的 OOM 调整值和

线程调度组将按照与 BRS 中剃刀的 Service 进程相同的规则进行更改。因此，Provider 进程不会被 RRT，KBP 和 KAP 杀死。

实现：为了将自身与 Provider 进程建立连接，应用程序进程调用定义在 ContentResolver 类中的 acquireContentProviderClient 或 acquireUnstableContentProviderClient API。值得注意的是，如果 Provider 进程通过调用 ContentProviderClient 类提供的 APi（关闭或释放）断开连接，则 Provider 进程的优先级将不再受到其客户端进程的影响。

5 唤醒应用程序

由于表 2 中的任何方法都不能防止应用程序被 KSP 杀死，因此仅仅让应用程序单独存活是不够的。因此顽固应用程序在被停止后也会使用其他方法来唤醒自己。值得注意的是，如果停止运行的应用程序的某个组件已经启动，该应用程序视为被唤醒。

我们提出了 5 种唤醒顽固应用的方法，并列在了表 3 中。这些方法针对不同的场景，分为三类。首先，应用程序使用自己的 Service（例如粘性 Service）来唤醒自己（第 5.1 小节）。由于该方法无法重启 FSA 停止的普通应用程序，我们进一步提出了其他类型的方法。第二，应用程序利用系统组件来唤醒自己，例如，应用程序可以使用 Alarm 服务来周期性的启动其组件以唤醒自己。值得注意的是，安卓系统为应用程序使用系统组件设置了若干约束条件。例如，部分系统组件对于即时通信的应用程序来说是不可用的，不能即时的唤醒应用程序。因此，应用程序通常会采用第三类方法。第三，依赖其他应用程序唤醒。即已经被停止的应用等待来自其他应用的唤醒信号（例如广播等）。一旦收到信号，则启动其组件以响应信号，然后该应用将被唤醒。

表 3：唤醒应用程序的顽固手段

方法	是否支持 快应用	杀死应用进程的方法					
		RRT	FSA	KBP	KAP	KSP	LMK
CSS	√	●	◐	●	●	●	●
MSB	×	●	◐	●	●	●	●
LAS	√	●	◐	●	●	●	●
UJS	×	●	⊗	●	●	●	●
MAB	√	●	●	●	●	●	●

注：●、⬤和⊗表示的意义与表 2 一样。

5.1 利用应用组件

一个被停止运行的应用程序可以通过其特定的组件唤醒自己。

- **构建粘性 Service (Constructing Sticky Service, CSS)**

设计：粘性 Service 的独特特性使其适合执行长时间的任务^[10]。具体来说，如果运行粘性 Service 的进程在 Service 启动后(即它的 onStartCommand 回调已经执行)被杀死，安卓将重新启动该 Service。因此，被 RRT、KBP、KAP、KSP 或 LMK 拦截的顽固应用可以使用粘性 Service 唤醒自己。然而，该方法无法唤醒被 FSA 拦截的应用，因为 FSA 使得安卓系统忽略了应用的粘性 Service。

实现：为将一个已经启动的 Service 变为粘性 Service，该 Service 的 onStartCommand 回调必须返回下面的常量中的其中一个：START_STICKY_COMPATIBILITY，START_STICKY，或 START_REDELIVER_INTENT。若使用前两个常量，Service 可以被安卓框架多次启动，而如果使用最后一个常量，Service 将只能重启一次。为确保 onStartCommand 的回调会被安卓框架调用，应用程序应该使用 startService API 启动粘性 Service。值得注意的是，重启的 Service 默认是一个后台 Service，如果再后台运行超过一分钟，该 Service 将被 AMS 停止^[1]。为解决这个问题，顽固应用程序可以使用第 4.2 节中的 HFS 方法将重新启动的 Service 带到前台。

5.2 利用系统功能

利用特定的系统功能（例如系统服务和广播）可以唤醒顽固应用程序，我们详细阐释了 3 种基于 app-accessible 系统功能的顽固方法。

- **监听系统广播 (Monitoring System Broadcast, MSB)**

设计：应用程序可以使用广播接收器来监测系统事件^[36]。一旦其关注的事件发生，接收器将被启动。因此，被 RRT、KBP、KAP、KSP 或 LMK 停止的应用程序可以使用这种方法来唤醒自己。然而，由于 FSA 使得被停止的应用只接收携带标志 FLAG_INCLUDE_STOPPED_PACKAGES 的广播，而 FLAG_INCLUDE_STOPPED_PACKAGES 并不包含在应用所能接收到的系统广播中^[6]，因此 MSB 无法唤醒被 FSA 停止的普通应用。

实现：广播接收器可以在应用程序的 manifest 文件中声明，也可以通过框架 API

registerReceiver 进行动态代码注册。值得注意的是，如果应用已经被停止，只能启动其在 manifest 文件中声明的接收器，因为安卓系统只会检查应用的 manifest 文件以确定其接收器是否可以响应系统广播^[2]。因此，少数系统广播（例如 ACTION_SCREEN_ON）只能被动态注册的接收器接受，无法唤醒应用程序。此外，从安卓 8.0 开始，静态声明的接受者不再允许接收不指定目标接收器的隐式广播（例如大多数系统广播）^[1]。因此，只有不受这些限制的系统广播^[6]可以用于实现 MSB，我们在表 4 中列出了其中一部分。

表 4：实现 MSB 的部分系统广播

动作名称	系统事件
ACTION_BOOT_COMPLETED	设备完成启动
ACTION_LOCALE_CHANGED	设备位置变化
ACTION_MEDIA_MOUNTED	外部媒体挂载
ACTION_NEW_OUTGOING_CALL	外部呼叫即将被安置
ACTION_TIMEZONE_CHANGED	时区发生变化

- **利用 Alarm 服务（Leveraging Alarm Service, LAS）**

设计：AlarmManager 允许应用程序为执行某项任务而重复启动其组件。相应地，被 RRT、KBP、KAP、KSP、LMK 停止的应用程序可以利用此功能进行唤醒。然而，LAS 无法唤醒被 FSA 拦截的应用，因为 FSA 将 Alarm 服务中待处理的任务移除。

实现：为了重复启动一个应用程序组件，应用首先构造一个 PendingIntent 任务，该任务指定要启动的应用程序组件。然后，应用程序调用 AlarmManager 类中声明的 setRepeating 或 setInexactRepeating 函数来设置 AlarmManagerService 调度任务的时间间隔。

- **使用 Job Scheduling 服务（Using Job Scheduling Service, UJS）**

设计：从安卓 5.0 开始，应用程序可以请求安卓系统执行通过在应用的 JobService 组件中定义作业来定期执行作业。更具体的说，应用提交 JobInfo 对象来通知 JobSchedulerService 启动 JobService 组件并执行作业。因此，被 RRT、KBP、KAP、KSP 或 LMK 停止的应用可以使用 JobSchedulerService 重新启动其 JobService 组件并唤醒自身。但是，UJS 无法唤醒被 FSA 停止的应用，因为 FSA 取消了 JobSchedulerService 接受的调度作业。此外，由于 JobSchedulerService 无法通过包名和类名访问免安装应用的 JobService 组件^[7]，因此 UJS 无法唤醒免安装应用。

实现：使用 `JobInfo.Builder` 类，然后调用 `setPeriodic` 指定定期执行 `JobService` 组件中定义的作业的时间间隔。请注意，在安卓 8.0 之后，`setPeriodic` 将最小时间间隔（即存储在 `JobInfo` 对象的 `intervalMillis` 字段中的值）调整为 15 分钟，以优化电池使用。为了绕过这个限制，我们发现并利用了本地 `JobInfo` 对象和远程 `JobSchedulerService` 实例进行的时间间隔检查不一致的情况。具体来说，安卓仅在应用程序调用 `setPeriodic` 时在客户端检查时间间隔的有效性，而不是在 `JobSchedulerService` 将根据存储在 `JobInfo` 对象中的时间间隔安排作业时在服务器端检查。图 3 说明了一个概念验证(PoC)，可由顽固应用程序采用以绕过时间间隔限制。

```

1 // The value of the "intervalMillis" field of oldJob is no smaller than 15*60*1000.
2 public void exploit(JobInfo oldJob) {
3     // Step 1: Store the original JobInfo object using a Parcel container.
4     Parcel in = Parcel.obtain();
5     oldJob.writeToParcel(in, PARCELABLE_WRITE_RETURN_VALUE);
6     // Step 2: Revise the "intervalMillis" value to a smaller value.
7     Parcel out = Parcel.obtain();
8     // reviseParcel(*) is a customized method manipulates the buffer in the Parcel.
9     reviseParcel(in, PARCELABLE_WRITE_RETURN_VALUE, out);
10    // Step 3: Create the JobInfo object using the buffer of the Parcel container.
11    JobInfo newJob = JobInfo.CREATOR.createFromParcel(out);
12    jobScheduler.schedule(newJob);
13 }

```

图 3：调整 `JobInfo` 中存储的时间间隔。

5.3 依赖第三方应用

与顽固应用合谋的第三方应用程序可以通过显示指定组件的名称以启动顽固应用的 `Activity`、`Service` 或获取其 `Content Provider` 以唤醒顽固应用。由于根据限制指定的组件名称可以很容易的推断出顽固应用与合谋应用之间的关系，因此本文不研究这种方法，而是设计一种第三方应用隐式唤醒顽固应用的方法。

• 监听应用程序广播（Monitoring App Broadcast, MAB）

设计：应用程序可以使用在其 `manifest` 文件中声明的接收器监听来自第三方应用程序的广播以唤醒自身。与系统广播不同的是，一个被精心设计的隐式广播可以唤醒被 FSA 停止的应用程序。因此，被 6 种进程杀死方法中的任何一种所停止的应用程序都可以使用 MAB 来唤醒自己。

实现：按照设计，广播接收器需要在 `manifest` 文件中静态声明。其他应用发送的隐式广播需要满足 2 个要求：第一，广播需要绕过安卓 8.0 以后的隐式广播约束（即安卓不再允许应用发送隐式广播），第三方应用可以通过发送带有标志 `FLAG_RECEIVER_INCLUDE_BACKGROUND` 的广播来规避。第二，被 FSA 停止的应用需要能够接收到隐式广播，为此，第三方应用需要让隐式广播携带标志

FLAG_INCLUDE_STOPPED_PACKAGES。

6 检测顽固应用程序

我们对于这些顽固手段的在野流行度很感兴趣，为了进行大规模的研究，我们开发了一个叫做顽固检测器的新工具以检查应用程序是否使用了在第 4 节和第 5 节中描述的顽固手段。在第 6.1 小节简要介绍顽固检测器后，我们将在第 6.2~6.4 小节中详细介绍顽固检测器的各个模块。

6.1 总览

如图 4 所示，顽固检测器包含三个模块：manifest 解析器，字节码分析器和检测器。它接收应用程序的 APK 文件输入，并输出该应用程序使用的顽固方法（如果有）。

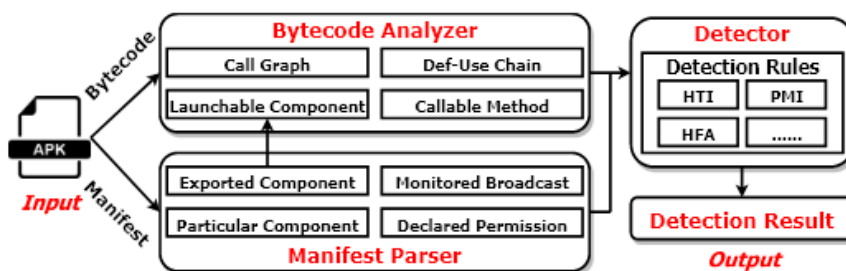


图 4：顽固检测器流程总览

- manifest 解析器（第 6.2 小节）：该模块解析应用程序的 manifest 文件，以收集相关的暴露的应用程序组件的信息，以便于后续对应用程序的 Dex 文件进行静态分析。为了获取检测顽固方法所需的信息，该模块还检索应用程序监控的广播，查找具有特殊属性的应用程序组件，并收集声明的权限。
- 字节码分析器（第 6.3 小节）：该模块首先通过分析应用程序的 Dex 文件构建调用图并建立定义-使用链。然后，它查找无法启动的应用程序组件，以修剪由于保守的图构建策略而导致的无效调用图边^[12]，该策略假设所有应用程序组件都是可启动的。随后，它收集修剪后的调用图中表示方法的节点，包括应用程序定义的方法和框架 API，以识别顽固方法。
- 检测器（第 6.4 小节）：该模块以 manifest 解析器和字节码分析器的结果作为输入，根据第 4 和第 5 节中描述的顽固方法定义规则，以发现应用程序使用的顽固方法。

6.2 解析 Manifest 文件

我们从 manifest 文件中收集以下信息：

- 暴露的应用程序组件。为了收集可以被外部启动的应用程序组件，顽固检测器检阅每个声明的应用程序组件的 `exported` 属性，并将暴露的应用程序组件储存到集合 S_{exported} 中。注意，如果一个应用程序组件包含 `intent-filter`，它就可以接受外部 `intent` 意图，所以因此顽固检测器也将这样的组件加入集合 S_{exported} 中。
- 监听的系统广播。为了获得由静态声明的接收器监控的广播，顽固检测器检查了每个广播接收器的意图过滤器，并将系统广播和应用程序广播分别为 2 个集合，分别为 S_{syscast} 和 S_{appcast} 。
- 特殊应用程序组件。要查找与顽固方法相关的应用程序组件，顽固检测器解析了每个应用程序组件的属性。确切地说，带有 `process` 属性的 `Service` 或 `Content Provider` 可以运行在单独的进程中，这样的组件将被储存到 S_{process} 集合中。此外，具有 `taskAffinity` 的 `Activity` 将把 `Activity` 放在单独的 `Activity` 任务中，将保存到 S_{affinity} 中。如果 `Activity` 启用其 `excludeFromRecents` 属性，则使相应的 `Activity` 任务与最近的任务项目不关联，则该 `Activity` 将记录到集合 S_{recents} 。
- 声明的权限。为了识别需要特定权限的顽固方法（例如，`COW`），顽固检测器在清单文件中解析了 `uses-permission` 标签，并将声明的权限存储到 $S_{\text{permission}}$ 集合中。

6.3 分析字节码

为了促进第 6.4 小节中的检测，顽固检测器执行静态分析以构建准确的函数调用图，构建定义-使用链，并识别可启动组件以及可调用的方法。

- 函数调用图：为了找到识别 `HFA` 和 `HFS` 的执行路径，顽固检测器首先使用 `FlowDroid`^[12] 来构建应用程序的函数调用图，然后修剪无效的边。

给定一个应用程序，`FlowDroid` 创建了一个入口点，并将其与应用程序组件的回调联系起来，假设所有应用程序组件都是可启动的。也就是说，可以始终执行应用程序组件的回调。然后，`FlowDroid` 找到可从回调构建函数调用图的方法。但是，由于无法启动某些应用程序组件，因此函数调用图包含无效的边，这可能会给顽固应用程序的检测带来误报。

为了修剪无效边，顽固检测器查找可启动的应用程序组件，保留从可启动应用程序组件的回调可达的边，并移除其他边（即无效边）。通常，可启动的应用程序组件有三种类型：

- (1) 暴露的应用程序组件，可以从外部（例如用户或其他应用程序）启动；
- (2) 可以由暴露

的应用程序组件启动的应用程序组件；(3) 可以由可启动的应用程序组件启动的应用程序组件。因此，由于暴露的应用程序组件始终是可启动的，顽固检测器查找可启动的非暴露的应用程序组件以修剪无效边。

修剪无效边的过程包括三个步骤。第一步，顽固检测器查找可以由其他应用程序组件启动的非暴露的应用程序组件。具体来说，顽固检测器定位用于启动应用程序组件的框架 API（例如表 6 中列出的 API）。然后，顽固检测器对传递给这些 API 的 Intent 对象进行定义-使用分析，因为这些 Intent 对象的使用包含启动者（即启动非暴露的应用程序组件的应用程序组件）的名称以及要启动的非暴露的应用程序组件。第二步，为了确保被启动的非暴露的应用程序组件是可启动的应用程序组件，顽固检测器检查它们是否由可启动的应用程序组件启动。更具体地说，顽固检测器检查非暴露的应用程序组件的启动者是否是可启动的应用程序组件。如果是，则这些非暴露的应用程序组件是可启动的应用程序组件。第三步，为了修剪无效边，顽固检测器从已识别的可启动应用程序组件的回调开始遍历调用图，然后移除无法访问的边。

表 6：部分启动应用程序组件的 API

组件	框架 API
Activity	Context.startActivity(Intent) Activity.startActivity(Intent) PendingIntent.getActivity(Context,int,Intent,int)
Service	Context.startService(Intent) PendingIntent.getService(Context,int,Intent,int) Context.bindService(Intent,ServiceConnection,int)
Broadcast Receiver	Context.sendBroadcast(Intent) PendingIntent.getBroadcast(Context,int,Intent,int) Context.registerReceiver(BroadcastReceiver,IntentFilter)

- 定义-使用链：为了检测 COW、BRS 和 ACP，顽固检测器构建了选定变量的定义-使用链^[18]。更具体地说，为了获取创建的覆盖窗口的类型以检测 COW，顽固检测器构建了将被赋值给 Window 的 setAttributes 方法的参数或 Window 的 setType 方法的类型参数的变量的定义-使用链。为了找到不会被解绑的应用程序 Service 以检测 BRS，顽固检测器

构建了 ServiceConnection 对象的定义-使用链，这些对象将被传递给 bindService API 或 unbindService API。为了找到不会被释放或关闭的 Content Provider 以检测 ACP，顽固检测器构建了 ContentProviderClient 对象的定义-使用链，这些对象要么是 acquireContentProviderClient 的返回对象，要么是调用 ContentProviderClient 的 release/close 方法的基础对象。

此外，为了找到使用特殊 Intent 标志启动的应用程序组件，这对于识别 PMI、CSS 和 MAB 至关重要，顽固检测器构建了传递给用于启动应用程序组件的框架 API 的 Intent 对象的定义-使用链。顽固检测器将使用 FLAG_ACTIVITY_NEW_TASK 标志启动的 Activity 存储到集合 S_{task} 中，将使用 startService 启动的 Service 保存到集合 $S_{startService}$ 中，并将使用 FLAG_RECEIVER_INCLUDE_BACKGROUND 和 FLAG_INCLUDE_STOPPED_PACKAGE 标志发送的广播放入集合 $S_{broadcast}$ 中。

- 可启动组件：要查找用于识别 HFA 和 HFS 的可启动应用程序组件，顽固检测器检查了函数调用图，以获取方法签名包含应用程序组件的可启动应用程序组件的回调。顽固检测器将可启动的应用程序组件保存到集合 $S_{launchable}$ 中。
- 可调用方法：为了获取用于识别 HTI、COW、LAS 和 UJS 的可调用方法，顽固检测器将调用图中的节点存储到集合 $S_{callable}$ 中，每个节点对应一个可以从可启动应用程序组件的回调中访问到的方法。

6.4 检测顽固手段

顽固检测器根据表 5 中总结的规则标识了各种顽固的方法。我们在下文中做出解释。

表 5：检测顽固方法的核心规则

行为	检测规则
HTI	$(S_{exported} \cap S_{recents} \neq \emptyset) \vee ((AppTask.setExcludeFromRecents \in S_{callable}) \wedge (args[0]=true))$
PMI	$(S_{affinity} \cap S_{task}) \neq \emptyset$
HFA	$\exists receiver, (receiver \in S_{launchable}) \wedge ((path(onReceive, *.startActivity) \neq \emptyset) \vee (path(onReceive, PendingIntent.getActivity) \neq \emptyset))$
HFS	$\exists service, (service \in S_{launchable}) \wedge ((path(onCreate, startForeground) \neq \emptyset) \vee (path(onBind, startForeground) \neq \emptyset) \vee (path(onStartCommand, startForeground) \neq \emptyset))$
COW	$(SYSTEM_ALERT_WINDOW \in S_{permission}) \wedge (((Window.setAttributes \in S_{callable}) \wedge (args$

	$[0].type \geq \text{TYPE_PHONE}) \vee ((\text{Window.setType} \in S_{\text{callable}}) \wedge (\text{args}[0] \geq \text{TYPE_PHONE}))$
BRS	$\exists \text{serviceConnection}, (\text{serviceConnection} \in S_{\text{bind}}) \wedge (\text{serviceConnection} \notin S_{\text{unbind}})$
ACP	$\exists \text{contentProviderClient}, (\text{contentProviderClient} \in S_{\text{acquire}}) \wedge (\text{contentProviderClient} \notin (S_{\text{release}} \cup S_{\text{close}}))$
CSS	$\exists \text{service}, ((\text{service} \in S_{\text{exported}}) \vee (\text{service} \in S_{\text{startService}})) \wedge (\text{service} \in S_{\text{sticky}})$
MSB	$\exists \text{broadcast}, (\text{broadcast} \in \text{不受隐式广播约束的系统广播}) \wedge (\text{broadcast} \in S_{\text{syscast}})$
LAS	$(\text{AlarmManager.setInexactRepeating} \in S_{\text{callable}}) \vee (\text{AlarmManager.setRepeating} \in S_{\text{callable}})$
UJS	$\text{JobInfo\$Builder.setPeriodic} \in S_{\text{callable}}$
MAB	$((S_{\text{appcast}} \cap S_{\text{sndcast}}) \neq \emptyset) \wedge (S_{\text{sndcast}} \text{ 来自其他应用程序})$

注：语法 $\text{args}[\text{index}]$ 返回可调用方法的具体参数；语法路径 (src, tgt) 获取 src 方法开始，以 tgt 方法结束的调用链。

1. HTI: 顽固检测器检查了 S_{exported} 和 S_{recents} ，以检查暴露的 Activity 的 $\text{excludedFromRecents}$ 属性是否设置为 true 。如果找不到 Activity，顽固检测器将检查 S_{callable} 以查找是否可以调用 $\text{setExcludeFromRecents}$ 方法。如果以上任一条件被满足，这意味着该应用程序将隐藏其最近任务项，则检测到 HTI。
2. PMI: 如果一个 Activity 同时包含在 S_{affinity} 和 S_{task} 中，则检测到 PMI，因为启动该 Activity 将使应用程序拥有多个 Activity 任务，对应于多个最近任务项。
3. HFA: 顽固检测器分析调用图以确定可启动广播接收器的 onReceive 回调是否会调用 PendingIntent 的 startActivity 或 getActivity 来启动 Activity。如果发现，则检测到 HFA，因为当应用程序在后台运行时可以启动前台 Activity。
4. HFS: 顽固检测器通过检查 $S_{\text{launchable}}$ 中每个应用程序 Service 的回调（即 onCreate 、 onBind 和 onStartCommand ）来确定是否会在回调中调用 startForeground 。如果是，则表示应用程序将启动前台 Service，因此检测到 HFS。
5. COW: 顽固检测器首先检查 $S_{\text{permission}}$ 以确定应用程序是否请求了 $\text{SYSTEM_ALERT_WINDOW}$ 权限。如果是，顽固检测器分析传递给 Window 的 setAttributes 或 setType 方法的变量的定义-使用链，以获取应用程序创建的窗口类型。如果窗口类型的值不小于 TYPE_PHONE 的值，则检测到 COW，因为应用程序将创建一个覆盖窗口^[32]。
6. BRS: 检测 BRS 的核心是找到可以通过 bindService 连接的远程应用程序 Service，并且

这种连接永远不会通过 `unbindService` 断开。因此，顽固检测器分析传递给 `bindService` 或 `unbindService` 的 `ServiceConnection` 对象的定义链，并将找到的变量定义分别存储到集合 S_{bind} 和 S_{unbind} 中。如果这两个集合的交集不是空集，这意味着某些 `Service` 绑定永远不会被断开，顽固检测器然后分析传递给 `bindService` 的 `Intent` 对象的定义-使用链以找到绑定的 `Service`。如果该 `Service` 也包含在 $S_{process}$ 中，则远程 `Service` 将使用 `BRS` 来提升其进程优先级。

7. **ACP:** 顽固检测器通过找到不会释放或关闭的获取的 `ContentProviderClient` 对象来识别 `ACP`。我们将获取的对象（即 `acquireUnstableContentProviderClient` 或 `acquireContentProviderClient` 的返回对象）、关闭的对象（即调用 `ContentProviderClient` 的 `close` 方法的基础对象）和释放的对象（即调用 `ContentProviderClient` 的 `release` 方法的基础对象）的定义分别存储到集合 $S_{acquire}$ 、 S_{close} 和 $S_{release}$ 中。如果有一个对象包含在 $S_{acquire}$ 中但不包含在 $S_{release}$ 和 S_{close} 中，并且其对应的 `Content Provider` 包含在 $S_{process}$ 中，则我们找到一个将使用 `ACP` 提升其进程优先级的远程 `Content Provider`。
8. **CSS:** 顽固检测器检查每个应用程序 `Service` 的 `onStartCommand` 回调的返回值以找到粘性 `Service`，并将它们存储到集合 S_{sticky} 中。如果 S_{sticky} 和 $S_{startService}$ 的交集不是空集，则识别到 `CSS`，因为应用程序将使用 `startService` 启动粘性 `Service`。
9. **MSB:** 如果应用程序声明接收器来监控不受隐式广播约束的系统广播，我们认为此类应用程序实现了 `MSB`。
10. **LAS:** 为了识别 `LAS`，顽固检测器检查 $S_{callable}$ 以确定应用程序是否会调用 `AlarmManager` 类中声明的 `setRepeating` 或 `setInexactRepeating`。如果是，则表明应用程序将使用 `Alarm` 服务重新启动其组件。
11. **UJS:** 顽固检测器检查 $S_{callable}$ 以确定应用程序是否会调用 `JobInfo$Builder` 类中定义的 `setPeriodic` 方法。如果是，则应用程序的 `JobService` 组件将被重新启动，因此检测到 `UJS`。
12. **MAB:** 如果包含在 $S_{appcast}$ 中的应用程序广播出现在另一个应用程序的 $S_{sndcast}$ 中，则检测到 `MAB`，因为被检查的应用程序监听来自另一个应用程序的精心设计的广播。

7 评估

在本节中，我们通过回答以下 3 个研究问题来评估顽固方法的适用性，顽固检测器的性

能以及在野顽固应用程序的普遍性。

RQ1: 我们的顽固方法是否可以在最流行的安卓版本即安卓 5.1 到安卓 10.0 上使用?

RQ2: 顽固检测器的性能如何?

RQ3: 顽固的应用程序的普遍性如何?

数据集: 我们使用了 3 个数据集, 包括 data-3, data-2k 和 data-80k, 分别用于回答 3 个研究问题。为了回答 RQ1, 我们开发了一个普通应用程序和一个快应用, 这两个应用程序都实现了针对其应用程序类型的所有可用的顽固方法。我们还开发了一个用于验证 MAB 的辅助正常应用程序。为了回答 RQ2, 我们从 F - Droid^[4]下载 2080 个开源应用程序作为基准。为了回答 RQ3, 我们从 Google Play 上随机爬取了 81237 个应用程序^[5]。

7.1 RQ1: 顽固方法的适用性

方法: 为了评估我们顽固方法的适用性, 我们在由 Google 发布的各种版本的官方安卓系统上运行 data-3 数据集的每个应用程序。

结果: 我们所有的顽固方法都可以延长从 5.1 到 10.0 的官方安卓系统运行的应用程序时间。由于安卓 10.0 已在时间间隔检查中修补了漏洞 (第 5.2 小结中提到), 因此我们无法更改 JobsChedulerService 执行定期作业的最小时间间隔 (15 分钟), 因此 UJS 无法及时重新启动该应用程序。

对 RQ1 的回答: 我们所有的 12 种顽固方法都适用于流行的安卓系统, 范围从安卓 5.1 到安卓 10.0。但是, 安卓 10.0 限制了 UJS 的能力。

7.2 RQ2: 顽固检测器的性能

方法: 为了评估顽固检测器的性能, 我们首先使用顽固检测器对 data-2k 数据集中的应用进行分析, 然后手动分析应用的源代码, 判断其是否具有顽固行为。更具体地说, 我们在 data-2k 中手动分析了 500 个随机选择的应用程序, 其中 250 个被发现为顽固应用程序, 而剩下的 250 个是正常应用程序。

结果: 表 7 列出了检测结果, 其中#app 表示具有特定顽固行为的应用程序数量。由于 FlowDroid 在处理 data-2k 中的 9 个应用程序时因分析时间过长 (>30 分钟) 而失败, 顽固检测器成功分析了 2071 个应用程序, 并发现其中 632 个 (30.52%) 使用了顽固方法。在手动分析了 250 个检测到的顽固应用程序后, 我们没有发现任何误报。对于其他 250 个应用程序, 我们发现了 14 个漏报, 这些应用程序在构建调用图时导致 FlowDroid 抛出异常。因为它们

的调用图既不包含节点也不包含边，使得顽固检测器无法检测到它们的顽固方法（PMI 除外），从而将它们视为非顽固应用程序。因此，顽固检测器的精确度和召回率分别为 100% 和 94.70%。为了评估修剪调用图的效果（第 6.3 小节），我们将检测结果（ $\#app$ ）与未移除无效调用图边时的结果（ $\#app'$ ）进行了比较。具体来说， Δapp 显示了识别出的具有特定顽固行为的应用程序数量的增加（即 $\Delta app = \#app' - \#app$ ）。详细来说，如果不移除无效边，顽固应用程序将导致 53 个误报。此外，我们注意到超过 40% 的 $\#app'$ 对于 HFS 是误报，因为识别出的前台 Service 永远不会被启动。因此，移除无效调用图边对顽固检测器非常重要。

对 RQ2 的回答：顽固检测器能够以较高的准确率（100%）和召回率（94.7%）识别顽固应用程序。

表 7：在 data-2k 数据集中找到的顽固手段

行为	$\#app$	$\#app'$	Δapp	行为	$\#app$	$\#app'$	Δapp
HTI	118	118	+0	ACP	0	0	+0
PMI	10	13	+3	CSS	72	91	+19
HFA	2	2	+0	MSB	452	452	+0
HFS	139	240	+101	LAS	158	165	+7
COW	11	14	+3	UJS	19	19	+0
BRS	0	0	+0	MAB	0	0	+0

7.3 RQ3: 顽固应用的流行度

方法：为了检测实际应用中的顽固应用程序并调查顽固方法的流行度，我们将顽固检测器应用于分析从 Google Play 抓取的 data-80k 中的应用程序。

结果：顽固应用程序分析一个应用程序的平均时间为 45 秒。表 8 列出了总体结果，其中 Ratio 是具有特定顽固方法的顽固应用程序的比例。结果显示，在我们的数据集中，Google Play 应用程序中有 17,410 个（约 21%）使用了顽固方法，因此顽固应用程序在应用程序生态系统中非常普遍。此外，MSB、LAS、CSS、HFS 和 HTI 是最流行的顽固方法，大约 70% 的顽固应用程序实现了 MSB。有趣的是，ACP 和 MAB 在 data-2k 和 data-80k 中未被发现。因此，我们发现了两种新的方法，应用程序可以使用这些方法来保持其进程存活并唤醒自己。由于一个顽固应用程序可能采用多种顽固方法，我们还分析了每个顽固应用程序中使用的顽

固方法的数量，结果如表 9 所示。具体来说，超过 35%的顽固应用程序采用了多种顽固方法，但很少有顽固应用程序采用超过四种顽固方法。我们还研究了不同应用程序类别中顽固应用程序的分布情况，结果列在表 10 中。结果显示，个性化类别中超过 40%的应用程序使用了顽固方法。此外，通信、新闻与杂志以及工具等类别中，超过 30%的应用程序是顽固应用程序。

对 RQ3 的回答：在我们的数据集中，大约 21%的 Google Play 应用采用了顽固的方法。排在前 5 位的是 MSB、LAS、CSS、HFS 和 HTI。ACP 和 MAB 尚未得到广泛应用。此外，顽固方法在个性化、通信、新闻周刊和工具等类别的应用程序中被广泛使用。

表 8：在 data-80k 数据集中，使用顽固手段的应用程序

行为	#app	比例	行为	#app	比例
HTI	1878	10.79%	ACP	0	0.00%
PMI	424	2.44%	CSS	1933	11.14%
HFA	125	0.72%	MSB	12413	71.30%
HFS	2243	12.88%	LAS	6452	37.06%
COW	264	1.52%	UJS	130	0.75%
BRS	21	0.12%	MAB	0	0.00%

表 9：应用程序中使用的顽固方法的数量

data-2k (632)			data-80k (17410)		
数量	#app	比例	数量	#app	比例
1	363	57.44%	1	11213	64.41%
2	205	32.44%	2	4444	25.53%
3	50	7.91%	3	1375	7.90%
4	12	1.90%	4	272	1.56%
5	2	0.31%	5	75	0.43%
6	0	0.00%	6	31	0.17%

表 10：顽固应用在各种应用类别中的比例

类别	比例	类别	比例	类别	比例
----	----	----	----	----	----

个性化	40.26%	社交	25.26%	娱乐	18.26%
通讯	39.44%	视频播放	24.91%	摄影	18.04%
新闻杂志	31.80%	生活	22.33%	金融	17.31%
工具	31.47%	健康	22.17%	教育	14.39%
音乐音频	29.40%	商务	21.87%	书籍	13.92%
购物	27.60%	旅行	20.02%	游戏	10.10%

8 对有效性的威胁

对顽固检测器的外部有效性的威胁来自两个方面。首先，一些特殊的应用程序样本可能会对顽固检测器的性能产生负面影响。更确切地说，由于静态分析的内在问题，顽固检测器无法处理采用高级混淆或加壳技术以防止对应用程序的字节码进行分析的应用程序。为了解决此问题，我们可以使用 DeObfuscator（例如 Tiro^[29]）和拆壳器（例如 PackerGrind^[30,31]）恢复受保护的字节码。此外，某些应用程序可能会使用原生代码来调用与构造应用程序的函数调用图图和检测顽固方法相关的方法（例如，起始性）。FlowDroid 可能会错过它们，因为它无法分析应用程序的原生代码。为了减轻此问题，我们可以使用 JN-SAF^[28]来获取应用程序本机代码中实现的方法调用。其次，尽管我们设计了 12 种顽固的方法，但我们可能会错过在野应用采用的其他一些顽固的方法。然而，这是对顽固行为的最全面研究，现有研究从未提及拟议中的顽固方法中的一半以上。

9 讨论

- **顽固方法的适用性：**第 7.1 小节中，我们使用 Google 发布的不同版本的安卓系统评估了顽固方法的适用性。但是，为了实现延长电池寿命的目的，移动设备制造商将向其定制的 Android 系统引入自己的杀活方法，这可能会降低提出的顽固方法的有效性。在未来的工作中，我们将进一步评估使用不同移动设备供应商的自定义安卓系统的顽固方法的适用性和有效性。
- **故意的顽固行为：**在本文中，我们没有区分对手故意实施的顽固行为，以及良性开发人

员不小心产生的行为。在将来的工作中，我们打算区分这两种顽固的行为。例如，如果应用程序在检查该进程的存活状态后重新启动了一个死的应用程序，我们可以考虑这种过程意味着顽固的行为是故意执行的。此外，如果保持应用程序流程的行为不涉及任何用户交互，我们也可以将其视为有意执行的顽固行为。

10 相关工作

本部分介绍了与寻找顽固应用最相关的研究。首先，最近的研究识别了不寻常的应用程序行为。Shan 等人^[24]提出了一种静态分析工具来检测自隐藏行为，防止应用程序被用户注意。OverlayChecker^[32]在仿真器中运行应用程序，以检查应用程序在运行时是否会启动一个覆盖窗口。DDAX^[25]进行静态分析，以发现滥用设备管理员这一安卓系统提供的敏感功能的应用程序。Shao 等人^[26]开发了一个分析应用程序生命周期的工具，可以用来检测一些顽固的方法，例如，HFS，CSS，MSB，LAS 和 UJS。然而，它不能检测我们提出的其他顽固方法。此外，我们还率先揭示了现有过程消灭方法的局限性，并将其用于设计顽固方法。值得注意的是，文献中提出的 12 种方法中有 7 种没有提及^[26]。

其次，研究人员研究了影响 Android 应用能耗的因素。EnergyPatch^[14]和 μ Droid^[19]指出，对高能耗硬件组件的不适当操作会导致应用程序表现出较差的能耗。HOT-PEPER^[15]的研究表明，应用的不良实施行为会对其能耗产生负面影响。Oliveira 等人^[21]和 Chowdhury 等人^[16]发现用于开发应用程序的编程语言和运行时执行日志会影响它们的能耗。GEMMA^[20]对应用程序使用的颜色进行了优化，以减少智能手机显示器上的能量消耗。然而，他们都没有提到过顽固行为会影响应用程序的能耗。

11 总结

我们对顽固的应用程序和顽固的方法进行了首次系统调查。通过揭示和利用现有的杀死进程方法的局限性，我们设计了 12 种实用的顽固方法，用于保活应用程序进程或唤醒停止的应用程序，该应用程序可用于 5.1-10.0 的流行安卓版本。为了自动识别顽固的方法的存在，我们开发了顽固检测器，可以准确，快速检测我们的顽固方法。广泛的实验结果表明，顽固检测器实现了高精度和高召回率。通过将顽固检测器应用于从 Google Play 获取的 8 万多个

应用程序中，我们观察到大约 21% 的应用程序采用了顽固的方法，这表明顽固应用程序在野外普遍存在。为了帮助用户了解顽固的行为，应用商店可能会要求开发人员在其应用程序的隐私政策中描述此类行为^[34,35]，并检查这些应用程序是否已经披露了其所有顽固的行为^[33]。

12 致谢

我们感谢匿名评论者的有益评论。本研究得到香港 RGC 项目((No.152279/16E , 152223/17E , CityU C1008-16G))、国家自然科学基金项目((No. 61702045 , No. 61872438))、浙江省创新创业领军团队引进计划项目(2018R01005)、浙江省重点研发计划项目(2019C03133)和国家自然科学基金项目((编号 1953893、1953813 和 1951729))的部分支持。

参考文献

- [1] 2020. Background Execution Limits. <https://developer.android.com/about/versions/oreo/background>.
- [2] 2020. Broadcasts Overview. <https://developer.android.com/guide/components/broadcasts>.
- [3] 2020. Don't kill my app! <https://dontkillmyapp.com/>.
- [4] 2020. F-Droid. <https://f-droid.org>.
- [5] 2020. Google Play Store. <https://play.google.com/store>.
- [6] 2020. Implicit Broadcast Exceptions. <https://developer.android.com/guide/components/broadcast-exceptions>.
- [7] 2020. Launch Instant App. <https://developers.google.com/android/reference/com/google/android/gms/instantapps/Launcher>.
- [8] 2020. Recents Screen. <https://developer.android.com/guide/components/activities/recents>.
- [9] 2020. Smartphone users still want long-lasting batteries more than shatterproof screens. <https://today.yougov.com/topics/technology/articles-reports/2018/02/20/smartphone-users-still-want-longer-battery-life>.
- [10] 2020. Sticky Service. https://developer.android.com/reference/android/app/Service#START_STICKY.
- [11] 2020. Understand Tasks and Back Stack. <https://developer.android.com/guide/components/activities/tasks-and-back-stack>.
- [12] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Oceau, and Patrick McDaniel. 2014. FlowDroid: Precise Context, Flow, Field, Object-sensitive and Lifecycle-aware Taint Analysis for Android Apps. In Proc. PLDI.
- [13] Kathy Wain Yee Au, Yi Fan Zhou, Zhen Huang, and David Lie. 2012. PScout:

- Analyzing the Android Permission Specification. In Proc. CCS.
- [14] Abhijeet Banerjee, Lee Kee Chong, Clément Ballabriga, and Abhik Roychoudhury. 2018. EnergyPatch: Repairing Resource Leaks to Improve Energy-Efficiency of Android Apps. *IEEE Transactions on Software Engineering* (2018).
- [15] Antonin Carrette, Mehdi Adel Ait Younes, Geoffrey Hecht, Naouel Moha, and Romain Rouvoy. 2017. Investigating the energy impact of Android smells. In Proc. SANER.
- [16] Shaiful Chowdhury, Silvia Di Nardo, Abram Hindle, and Zhen Ming Jiang. 2018. An exploratory study on assessing the energy impact of logging on Android applications. *Empirical Software Engineering* (2018).
- [17] Yanick Fratantonio, Chenxiong Qian, Simon P Chung, and Wenke Lee. 2017. Cloak and Dagger: From Two Permissions to Complete Control of the UI Feedback Loop. In Proc. S&P.
- [18] Mary Jean Harrold and Mary Lou Soffa. 1994. Efficient Computation of Interprocedural Definition-Use Chains. *ACM Trans. Program. Lang. Syst.* (1994).
- [19] Reyhaneh Jabbarvand and Sam Malek. 2017. μ Droid: An Energy-Aware Mutation Testing Framework for Android. In Proc. FSE.
- [20] Mario Linares-Vásquez, Carlos Bernal-Cárdenas, Gabriele Bavota, Rocco Oliveto, Massimiliano Di Penta, and Denys Poshyvanyk. 2017. GEMMA: Multi-objective Optimization of Energy Consumption of GUIs in Android Apps. In Proc. ICSE.
- [21] Wellington Oliveira, Renato Oliveira, and Fernando Castor. 2017. A Study on the Energy Consumption of Android App Development Approaches. In Proc. MSR.
- [22] Chuangang Ren, Peng Liu, and Sencun Zhu. 2017. WindowGuard: Systematic Protection of GUI Security in Android. In Proc. NDSS.
- [23] Chuangang Ren, Yulong Zhang, Hui Xue, Tao Wei, and Peng Liu. 2015. Towards Discovering and Understanding Task Hijacking in Android. In Proc. USENIX Security.
- [24] Zhiyong Shan, Iulian Neamtiu, and Raina Samuel. 2018. Self-hiding Behavior in Android Apps: Detection and Characterization. In Proc. ICSE.
- [25] Zhiyong Shan, Raina Samuel, and Iulian Neamtiu. 2019. Device Administrator Use and Abuse in Android: Detection and Characterization. In Proc. MobiCom.

- [26] Yuru Shao, Ruowen Wang, Xun Chen, Ahemd M. Azab, and Z. Morley Mao. 2019. A Lightweight Framework for Fine-Grained Lifecycle Control of Android Applications. In Proc. EuroSys.
- [27] Yulei Sui, Ding Ye, and Jingling Xue. 2014. Detecting memory leaks statically with full-sparse value-flow analysis. IEEE Transactions on Software Engineering (2014).
- [28] Fengguo Wei, Xingwei Lin, Xinming Ou, Ting Chen, and Xiaosong Zhang. 2018. JN-SAF: Precise and Efficient NDK/JNI-Aware Inter-Language Static Analysis Framework for Security Vetting of Android Applications with Native Code. In Proc. CCS.
- [29] Michelle Y Wong and David Lie. 2018. Tackling runtime-based obfuscation in Android with TIRO. In Proc. USENIX Security.
- [30] Lei Xue, Xiapu Luo, Le Yu, Shuai Wang, and Dinghao Wu. 2017. Adaptive unpacking of Android apps. In Proc. ICSE.
- [31] L. Xue, H. Zhou, X. Luo, L. Yu, D. Wu, Y. Zhou, and X. Ma. 2020. PackerGrind: An Adaptive Unpacking System for Android Apps. IEEE Transactions on Software Engineering (2020).
- [32] Yuxuan Yan, Zhenhua Li, Qi Alfred Chen, Christo Wilson, Tianyin Xu, Ennan Zhai, Yong Li, and Yunhao Liu. 2019. Understanding and Detecting Overlay-based Android Malware at Market Scales. In Proc. MobiSys.
- [33] L. Yu, X. Luo, J. Chen, H. Zhou, T. Zhang, H. Chang, and H. Leung. 2019. PPChecker: Towards Accessing the Trustworthiness of Android Apps' Privacy Policies. IEEE Transactions on Software Engineering (2019).
- [34] Le Yu, Tao Zhang, Xiapu Luo, and Lei Xue. 2015. AutoPPG: Towards Automatic Generation of Privacy Policy for Android Applications. In Proc. SPSM.
- [35] L. Yu, T. Zhang, X. Luo, L. Xue, and H. Chang. 2017. Towards Automatically Generating Privacy Policy for Android Apps. IEEE Transactions on Information Forensics and Security (2017).
- [36] Yajin Zhou and Xuxian Jiang. 2012. Dissecting Android Malware: Characterization and Evolution. In Proc. S

Demystifying Diehard Android Apps

Hao Zhou
The Hong Kong Polytechnic
University
Hong Kong, China
cshaoz@comp.polyu.edu.hk

Xiapu Luo*
The Hong Kong Polytechnic
University
Hong Kong, China
csxluo@comp.polyu.edu.hk

Haoyu Wang
Beijing University of Posts and
Telecommunications
Beijing, China
haoyuwang@bupt.edu.cn

Yutian Tang
ShanghaiTech University
Shanghai, China
csytang@ieee.org

Ting Wang
Pennsylvania State University
Pennsylvania, USA
inbox.ting@gmail.com

Yajin Zhou
Zhejiang University
Hangzhou, China
yajin_zhou@zju.edu.cn

Lei Xue
The Hong Kong Polytechnic
University
Hong Kong, China
cslxue@comp.polyu.edu.hk

ABSTRACT

Smartphone vendors are using multiple methods to kill processes of Android apps to reduce the battery consumption. This motivates developers to find ways to extend the liveness time of their apps, hence the name diehard apps in this paper. Although there are blogs and articles illustrating methods to achieve this purpose, there is no systematic research about them. What's more important, little is known about the prevalence of diehard apps in the wild.

In this paper, we take a first step to systematically investigate diehard apps by answering the following research questions. First, why and how can they circumvent the resource-saving mechanisms of Android? Second, how prevalent are they in the wild? In particular, we conduct a semi-automated analysis to illustrate insights why existing methods to kill app processes could be evaded, and then systematically present 12 diehard methods. After that, we develop a system named DiehardDetector to detect diehard apps in a large scale. The experimental result of applying DiehardDetector to more than 80k Android apps downloaded from Google Play showed that around 21% of apps adopt various diehard methods. Moreover, our system can achieve high precision and recall.

CCS CONCEPTS

• **Software and its engineering** → **Software verification and validation.**

*The corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ASE '20, September 21–25, 2020, Virtual Event, Australia

© 2020 Association for Computing Machinery.

ACM ISBN 978-1-4503-6768-4/20/09...\$15.00

<https://doi.org/10.1145/3324884.3416637>

ACM Reference Format:

Hao Zhou, Haoyu Wang, Yajin Zhou, Xiapu Luo, Yutian Tang, Lei Xue, and Ting Wang. 2020. Demystifying Diehard Android Apps. In *35th IEEE/ACM International Conference on Automated Software Engineering (ASE '20)*, September 21–25, 2020, Virtual Event, Australia. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3324884.3416637>

1 INTRODUCTION

Battery life is one of major concerns of users. A recent survey showed that “41% of US smartphone users say a longer battery life is the design feature they want most.” [9] Accordingly, smartphone vendors deploy *resource-saving mechanisms* to optimize the battery usage to kill background app processes [3].

However, from another perspective, app developers intend to increase the active time (liveness) of their apps. By doing so, they can maintain an inactive experience with users, such as providing timely services (e.g., sending notifications), making profit (e.g., pushing advertisements), etc. Indeed, recent years have witnessed many apps (diehard apps) employ various approaches to bypass resource-saving mechanisms. These approaches, which can prolong diehard apps' lifetime, are called *diehard methods*.

The diehard methods can be roughly divided into 2 categories according to their purposes: (1) keeping apps alive; (2) waking up apps. Shao et al. [26] manually studied 23 diehard apps and found a few diehard methods, most of which aim at waking up apps. However, the community still lacks a deep understanding of diehard methods and diehard apps.

Our work In this paper, we take a first step to systematically investigate diehard apps by answering 3 research questions: (1) Why could existing methods to kill app processes be circumvented? (2) How do diehard apps keep themselves alive or be waken up? (3) Do diehard behaviors widely exist in apps in the wild? Answers to these questions can facilitate the development of effective resource-saving mechanisms to benefit users (with a battery-efficient system) and keep the app ecosystem healthy by discovering diehard apps.

To this end, we first perform a systematic study on existing methods to kill app processes and analyze why these methods could be evaded (in §3). It is not easy to locate these methods and figure out corresponding evasion approaches because Android system has more than 10 millions lines of code and it is still evolving, let alone the lack of detailed documentation for those methods. We solve this challenge by taking a semi-automated approach that uses static code analysis to extract mostly relevant source code and then manually inspect it to understand the internal mechanisms.

Second, leveraging the observations obtained in the previous step, we systematically propose 12 diehard methods. Among them, 7 methods can keep the app alive (in §4), and 6 of them have *not* been reported before. The remaining 5 approaches can wake up diehard apps (in §5), and 1 of them is *not* mentioned in the previous work [26]. To evaluate the effectiveness of these methods, we develop 2 diehard apps (a normal app and an instant app). The extensive experiments on popular Android versions, ranging from Android 5.1 to Android 10.0, show that all of our methods can prolong apps' liveness time.

Third, to detect diehard apps in the wild, we develop DiehardDetector, a new detection tool based on the 12 diehard methods. We carefully design DiehardDetector to make it achieve low false positive and false negative rates and be able to inspect a large number of Android apps. The evaluation results of DiehardDetector show that it is accurate (i.e., the precision and the recall of the results on detecting diehard apps are 100% and 94.70%, respectively) and efficient (i.e., 45 seconds for analyzing an app on average).

Fourth, we apply DiehardDetector to analyze more than 80k apps for identifying diehard behaviors. The experimental results show that around 21% of the apps from Google Play have diehard behaviors. Moreover, 2 proposed diehard methods have not been found in the apps under investigation. It suggests that we find 2 previously unknown methods to extend the liveness time of apps.

To engage the community, we release DiehardDetector and the apps involved in the evaluation at <https://github.com/moonZHH/DiehardDetector>.

In summary, this paper makes the following main contributions:

- To our best knowledge, we perform the first systematic investigation on diehard apps. In particular, we are the first to reveal why the resource-saving mechanisms can be circumvented and propose 12 diehard methods to keep apps alive and wake up apps.
- We develop DiehardDetector, a new tool that can identify diehard behaviors in Android apps accurately and efficiently.
- We conduct extensive experiments to evaluate DiehardDetector and examine the prevalence of diehard methods used by apps. The experimental results on over 80k apps from Google Play show that DiehardDetector can accurately identify the diehard behaviors of apps in a scalable manner. Moreover, around 21% of apps published in Google Play have diehard behaviors.

2 BACKGROUND

Android apps can be roughly classified into 2 categories: (1) system apps, which are usually pre-installed by smartphone vendors. Since such apps are protected by Android system from being killed, they rarely use diehard methods. (2) normal apps and instant apps, which are created by developers and downloaded by the smartphone users.

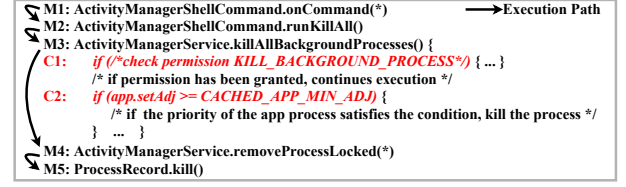


Figure 1: Finding the process-killing method, KAP.

Such apps may adopt diehard methods to prolong the liveness time for many purposes (e.g., pushing advertisements to make profits).

An app may have multiple processes. By default, an app's components (i.e., activities, services, broadcast receivers, and content providers) are running in the app's main process. However, each component can specify its process property in the manifest file to make it run in a separate process other than the main process. If all of an app's running processes are killed, the app is stopped.

Android framework provides 2 data structures, *ProcessRecord* and *TaskRecord*, to manage app processes. We leverage them to find out the methods for killing the app process or stopping the app. Note that the methods for stopping the app are special cases of methods for killing the app process because they are designed to kill all running processes associated with an app.

ProcessRecord: It is used to characterize each running app process by *ActivityManagerService* (AMS), one of the core services of Android system. Specifically, the *curAdj* (or *setAdj*) field and the *curSchedGroup* (or *setSchedGroup*) field of a *ProcessRecord* object hold an app process's priority values. There are several ways to notify AMS to create a *ProcessRecord* for an app, e.g., launching its activity, starting its service, sending a broadcast that can be responded by its receiver, accessing its content provider, etc.

TaskRecord: It is used to characterize an activity Task [11], which is a collection of activities that the user interacts with when performing a certain job. Meanwhile, in normal cases, a *TaskRecord* object of a recently accessed app corresponds to an item in the Recent-Task list [8]. Accordingly, a *TaskRecord* object links an activity task of a recently accessed app with a Recent-Task item.

3 METHODS FOR KILLING APP PROCESSES

To find the methods for killing app processes, we conduct a semi-automated investigation on the framework and the system applications (e.g., low memory killer, a.k.a LMK) of Android 9.0 in §3.1. We learn from this investigation that the priority of an app process affects whether it will be killed or not, and describe how Android system determines an app process's priority in §3.2. In §3.3, we disclose the internals of these process-killing methods, and point out the insights why they can be evaded, which are then exploited to design diehard methods (in §4 and §5).

3.1 Finding Process-killing Methods

Diehard apps employ diehard methods to prevent their processes from being killed. Thus we look for all process-killing methods to understand why they could be circumvented. There are 2 basic approaches to terminate an app process: (1) invoking the *kill* API defined in the *ProcessRecord* class, because Android framework uses *ProcessRecord* to represent a running app process; (2) using the *kill* function exported by *libc.so*, because native applications

or libraries can invoke this function that calls `syscall __NR_kill`. We adopt a semi-automated way to find other process-killing methods that will eventually call either of the above basic approaches. Starting with the basic approaches, we use static analysis to extract most relevant source code and then manually inspect them, as detailed in the following paragraphs. It is worth noting that this semi-automated way allows us to inspect the whole Android system and can be used to find new or changed process-killing methods in the future versions of Android.

Procedure: To find the process-killing methods in Android framework, we first construct the call graph of the framework by using PScout [13]. Then, we locate the `ProcessRecord.kill` API in the call graph and perform backward reachability analysis to get the execution paths that reach this API. Each execution path refers to a specific process-killing method. Moreover, to reveal the conditions for terminating the app process, we perform control-flow analysis on each execution path to locate the conditional statements that affect whether the `kill` API will be executed or not. Subsequently, we manually analyze the extracted conditional statements to disclose how to evade the process-killing methods.

As an example, Figure 1 shows the execution path and the necessary conditional statements of the process-killing method for killing all background processes (KAP, in §3.3). We find the execution path $M1 \rightarrow M2 \rightarrow M3 \rightarrow M4 \rightarrow M5$ that accesses the `ProcessRecord.kill` API (i.e., $M5$). Then, we locate the conditions (i.e., $C1$ and $C2$) that need to be satisfied to reach $M5$. After inspecting these conditional statements, $C2$ in particular, we learn that, the priority of the app process (i.e., the value stored in the `setAdj` of the corresponding `ProcessRecord` instance) determines whether this app process will be killed or not. If an app process's priority does not satisfy $C2$, this app process circumvents KAP.

Since the native applications in Android system can call the `kill` function exported by `libc.so`, to discover the process-killing methods in them, we build the call graph for each native application by using SVF [27], and locate the `kill` function. Once found, we reversely traverse the call graph from the `kill` function to get the execution paths that reach this function. Then, we manually analyze the source code of the methods in each execution path to investigate how to evade the process-killing methods.

Result: We find 6 process-killing methods (in §3.3) and 4 of them check the priority of the app process to decide whether it will be killed or not. Meanwhile, we notice that, after killing an app process, Android framework will call AMS's `updateOomAdjLocked` method to adjust the priority of current running app processes. To understand how AMS calculates the priority of an app process, we perform control-flow analysis on AMS's `computeOomAdjLocked` method, which will be called by `updateOomAdjLocked`. Specifically, since the local variables, `adj` and `schedGroup`, are used to update `curAdj` and `curSchedGroup` of the `ProcessRecord` instance, respectively, we extract the conditional statements that decide whether their values will be changed. Then, we manually analyze these statements to disclose why a process has a specific priority (in §3.2).

3.2 App Process Priority

Android determines the priority of an app process using the out-of-memory (OOM) adjustment and the thread scheduling group. In the following, we first introduce how the OOM adjustment of an app

process is set, which is relevant to the diehard methods proposed in §4.2, and then describe 2 thread scheduling groups that are relevant to the diehard methods proposed in §4.1. Table 1 summarizes the difference cases of app processes.

OOM Adjustment. An app process with high priority usually has a small OOM adjustment value and is less likely to be killed.

- * `FOREGROUND_APP_ADJ`: The process of the foreground app (e.g., **C01** and **C02**) is associated with this OOM adjustment.

- * `VISIBLE_APP_ADJ`: The app process (e.g., **C05**) that hosts visible activities is linked with this OOM adjustment.

- * `PERCEPTIBLE_APP_ADJ`: The app process (e.g., **C08**) that hosts perceptible components (e.g., foreground services connected with visible notifications) is mapped to this OOM adjustment.

- * `SERVICE_ADJ`: The priority of the app process (e.g., **C12**) that hosts the running services is depicted by this OOM adjustment.

- * `CACHED_APP_MIN_ADJ`: The app process (e.g., **C13**) that hosts invisible activities is assigned with this OOM adjustment.

Remark: If an app process satisfies multiple previous descriptions, its OOM adjustment is the one that has the smallest value. For example, the OOM adjustment of the app process, hosting a visible activity and running a service, is `VISIBLE_APP_ADJ`.

Thread Scheduling Group. AMS uses it to manage the liveness of a group of app processes. Normally, processes with the same priority tend to be organized in one thread scheduling group.

- * `SCHED_GROUP_TOP_APP`: This group contains the processes of foreground apps (e.g., **C01**), which have relatively high priority.

- * `SCHED_GROUP_BACKGROUND`: This group includes the processes of background apps (e.g., **C12**), which have relatively low priority.

3.3 Exploring Process-Killing Methods

In this section, we elaborate on the internals of the 6 process-killing methods discovered in §3.1, especially the conditions determining whether or not an app process will be killed, and point out the insights why the methods could be circumvented.

(1) Removing Recent-Task Item (RRT).

Usage: Users can swipe away an item presented in the Recent-Task list to stop a recently accessed app.

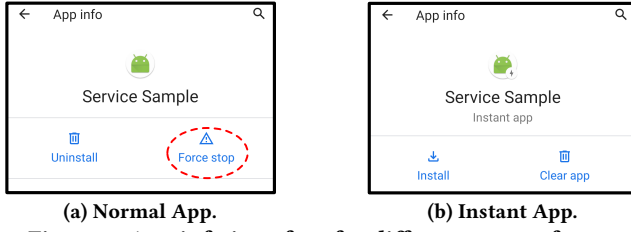
Internal: RRT consists of 2 major steps. First, AMS removes the `TaskRecord` object associated with the removed Recent-Task item from `ActivityStackSupervisor`'s `mRecentTasks` field, which is an array recording the `TaskRecord` object of each recently accessed app. Second, AMS uses `ActivityStackSupervisor` (ASS), which is the manager of activity stacks, to obtain the `ProcessRecord` objects associated with the app to be stopped. Once a `ProcessRecord` object is found, ASS checks whether the process hosts activities. If not, the process is the candidate to be killed. Otherwise, ASS checks whether the activities hosted by the process belong to the same activity task. If so, such process is the candidate as well. Before calling `ProcessRecord`'s `kill` method to terminate the candidate, ASS checks whether the process's thread scheduling group is `SCHED_GROUP_BACKGROUND`. If so, the process will be killed.

Remark: RRT cannot stop the app that does not have a Recent-Task item. Moreover, it kills neither the app process that has multiple activity tasks nor the app process whose thread scheduling group is not `SCHED_GROUP_BACKGROUND`.

(2) Force-stopping App (FSA).

Table 1: A summary of different cases of app processes.

OOM Adjustment	Value	Case	Scheduling Group
FOREGROUND_APP_ADJ	0	C01: The process has a foreground activity, and the device is awake.	SCHED_GROUP_TOP_APP (3)
		C02: The process has a foreground activity, but the device is sleeping.	SCHED_GROUP_BACKGROUND (0)
		C03: The process has a service, to which another process with such an OOM adjustment is bound.	SCHED_GROUP_DEFAULT (2)
		C04: The process has a provider, which is acquired by another process with such an OOM adjustment.	SCHED_GROUP_DEFAULT (2)
VISIBLE_APP_ADJ	100	C05: The process has a visible activity.	SCHED_GROUP_DEFAULT (2)
	1	C06: The process has a service, to which another process with such an OOM adjustment is bound.	SCHED_GROUP_DEFAULT (2)
	200	C07: The process has a provider, which is acquired by another process with such an OOM adjustment.	SCHED_GROUP_DEFAULT (2)
PERCEPTIBLE_APP_ADJ	200	C08: The process has a foreground service.	SCHED_GROUP_DEFAULT (2)
		C09: The process has an overlay window.	SCHED_GROUP_DEFAULT (2)
		C10: The process has a service, to which another process with such an OOM adjustment is bound.	SCHED_GROUP_DEFAULT (2)
		C11: The process has a provider, which is acquired by another process with such an OOM adjustment.	SCHED_GROUP_DEFAULT (2)
SERVICE_ADJ	500	C12: The process has a service, which was started within the last 30 minutes.	SCHED_GROUP_BACKGROUND (0)
CACHED_APP_MIN_ADJ	900	C13: The process has the stopped activity.	SCHED_GROUP_BACKGROUND (0)

**Figure 2: App info interface for different types of apps.**

Usage: Users can navigate to the App info interface and click the *Force stop* button to stop an app. A debugger can use the shell command, *am force-stop*, to achieve the same purpose.

Internal: FSA takes 3 steps to stop an app. First, AMS notifies the instance of *PackageManagerService* (PMS) to make the target app ignore the received broadcasts that were sent without carrying the flag, *FLAG_INCLUDE_STOPPED_PACKAGES*. Second, AMS finds the *ProcessRecord* objects associated with the app and calls *kill* method to terminate the processes. Third, AMS sends a broadcast to instruct the instances of *AlarmManagerService* and *JobSchedulerService* to clean up the pending tasks that were previously submitted by the app. Meanwhile, the broadcast will be received by *NotificationManagerService*, and the notifications linked with the target app will be dismissed accordingly.

Remark: In normal cases, FSA can kill all running processes of an app. However, we find that the instant app is an exception. As shown in Figure 2b, the App info interface of an instant app does not have the *Force stop* button. That is, FSA is unavailable for stopping instant apps, and the alive instant apps can wake up other apps.

(3) Killing Background Processes (KBP).

Usage: An app with the permission *KILL_BACKGROUND_PROCESSES* can terminate background app processes. More specifically, the app first obtains the instance of *ActivityManager* and then calls *ActivityManager*'s *killBackgroundProcesses* method to kill background app processes. A debugger can execute the shell command, *am kill*, to accomplish the same task.

Internal: The workflow of KBP contains 2 steps. First, AMS retrieves all alive *ProcessRecord* objects and examines the *setAdj* field of each to determine whether a candidate will be killed. Precisely, if the candidate's OOM adjustment value is no smaller than that of *SERVICE_ADJ*, it will be terminated. Second, AMS calls *kill* API in *ProcessRecord* to kill the target.

Remark: The first step of KBP implies it cannot kill the process, whose OOM adjustment value is smaller than that of *SERVICE_ADJ*.

(4) Killing All Background Processes (KAP).

Usage: A debugger can use the shell command, *am kill-all*, to kill all app processes running in the background, because this command makes AMS call its *killAllBackgroundProcess* method.

Internal: The workflow of KAP is similar to that of KBP, and the difference lies in the criteria for finding the target app processes. Precisely, KAP kills the app processes, whose OOM adjustment values are no smaller than that of *CACHED_APP_MIN_ADJ*.

Remark: The app process, whose OOM adjustment value is smaller than the value of *CACHED_APP_MIN_ADJ*, is excluded from the target of KAP, and thus it will not be killed by KAP.

(5) Killing Specific Process (KSP).

Usage: An app or a debugger can kill a specific process according to its PID by using the native *kill* function and the *kill* command, respectively. Note that an app can just kill its own processes while the privileged debugger does not have such constraints.

Internal: KSP calls the native *kill* function to terminate the target.

Remark: KSP cannot prevent the app from being waken up.

(6) Low Memory Killer (LMK).

Usage: If a running device's available RAM becomes insufficient, LMK's native daemon will kill a few selected app processes.

Internal: LMK includes 2 major steps. First, according to the remaining available RAM, it decides a threshold for the OOM adjustment value. Second, it lists the app processes, whose OOM adjustment values exceed the threshold. Among these processes, LMK uses the native *kill* function to kill the one that occupies the most amount of RAM. Then, LMK repeatedly executes these 2 steps until no process's OOM adjustment value exceeds the threshold.

Remark: The second step suggests that LMK rarely kills the app process with a high priority (i.e., a small OOM adjustment value).

4 KEEPING APPS ALIVE

In this section, by leveraging insights in §3.3, we propose 7 diehard methods, as listed in Table 2, to keep apps alive. Note that, if an app has a process running in the system, the app is alive.

4.1 Manipulating Recent-Task Item

We propose 2 diehard methods that manipulate the items in the Recent-Task list to evade RRT.

Table 2: Diehard methods for keeping apps alive.

Methods	Support Instant App	Method for Killing App Process					
		RRT	FSA	KBP	KAP	KSP	LMK
HTI	✓	●	○	⊗	⊗	⊗	⊗
PMI	✓	●	○	⊗	⊗	⊗	⊗
HFA	✓	⊗	⊗	●	●	⊗	○
HFS	✓	●	○	●	●	⊗	○
COW	✓	●	○	●	●	⊗	○
BRS	✓	●	○	●	●	⊗	○
ACP	✓	●	○	●	●	⊗	○

1 ● denotes that the method can prevent normal apps and instant apps from being killed; ○ denotes that the method can prevent normal apps or instant apps from being killed to a certain extent; ⊗ denotes that the method cannot prevent normal apps and instant apps from being killed.

• Hiding Recent-Task Item (HTI).

Design: Since RRT only stops the app that has an item in the Recent-Task list, the app can hide the item to evade RRT.

Implementation: To hide the Recent-Task item, an app can set the `excludeFromRecents` property of its main launchable activity to true in its manifest file or code. For the latter, the app first obtains the instance of `ActivityManager` and calls `getAppTasks` of `ActivityManager` to get the `AppTask` object that is bound with the Recent-Task item. Then, the app calls `setExcludeFromRecents` defined in the `AppTask` class to remove the Recent-Task item.

• Producing Multiple Recent-Task Items (PMI).

Design: Since RRT kills the process that organizes the hosted activities in an activity Task, to bypass it, the app can create multiple activity Tasks, which results in multiple Recent-Task items.

Implementation: To make Android create an additional activity Task for storing the activity, the `Intent` object used for launching the target activity should carry the flag `FLAG_ACTIVITY_NEW_TASK` [23]. Note that if the `taskAffinity` property of the source activity (i.e., intent sender) and the one of the target activity (i.e., intent receiver) are the same, Android will not create the new Task. To prevent this circumstance, the app should let the `taskAffinity` of the target activity be different from that of the source activity.

4.2 Escalating App Process Priority

Since RRT, KBP, KAP, and LMK kill app processes according to their priorities (i.e., the OOM adjustment or the thread scheduling group) as described in §3.3, we propose 5 methods to escalate an app process's priority for preventing the process from being killed or decreasing its probability of being killed.

• Holding Foreground Activity (HFA).

Design: As shown in Table 1, the app process (i.e., C01 or C02) holding the foreground activity has the smallest OOM adjustment value. Such processes will not be killed by KBP and KAP because they select app processes with lower priorities.

Implementation: To let a background app process hold a foreground activity, the app can declare or register a receiver that listens to system/app broadcasts. Once the app receives the broadcast, it will start an activity in the foreground. Diehard apps can further make HFA hard to be noticed. For example, the apps listen to the system broadcast `ACTION_SCREEN_OFF` and start the foreground activity when the device screen turns off or launch a very small activity.

• Hosting Foreground Service (HFS).

Design: The OOM adjustment for the app process (i.e., C08) that hosts a foreground service is `PERCEPTIBLE_APP_ADJ`, whose value is smaller than that of `SERVICE_ADJ` and `CACHED_APP_MIN_ADJ`. Hence, such app processes will not be killed by either KBP or KAP. Meanwhile, since the relevant thread scheduling group is `SCHED_GROUP_DEFAULT`, RRT cannot kill such app processes.

Implementation: An app first launches a service by creating an `Intent` object and calling `startService` or `bindService`. Then, to make the service foreground, in the `onCreate`, `onStartCommand`, or `onBind` callback of the service, the app creates a `Notification` object and passes it to the `startForeground` API, which will show a notification to inform the user that there is a foreground service.

• Creating Overlay Window (COW).

Design: An app process (i.e., C09) can launch an overlay window [17, 22], which will be rendered on top of any other app windows (e.g., interfaces of app activities), to adjust its thread scheduling group to `SCHED_GROUP_DEFAULT` and elevate its OOM adjustment to `PERCEPTIBLE_APP_ADJ`. Consequently, such app processes will not be killed by RRT, KBP, and KAP.

Implementation: Before Android 8.0, to make an app window become an overlay, the app calls `setAttribute` or `setType` defined in the `Window` class to specify the window's type with `TYPE_PHONE`, `TYPE_SYSTEM_ALERT`, `TYPE_SYSTEM_ERROR`, or `TYPE_TOAST`. For newly released Android systems (i.e., those published no earlier than Android 8.0), `TYPE_APPLICATION_OVERLAY` is usually adopted to construct the overlay window [32] because the aforementioned window types are deprecated.

• Binding Running Service (BRS).

Design: An app can run its services in separate processes (e.g., service processes) to reduce the resource occupation of its main process. Since the service process will usually not host any user perceptible components (e.g., UI), its OOM adjustment value is larger than that of `PERCEPTIBLE_APP_ADJ`. Hence, to prevent the service process from being killed, the app needs to escalate the priority of its service processes. After scrutinizing how AMS calculates and updates the priority of the app process, we find that the priority of the service process can be affected by the priorities of its client processes (i.e., C03, C06, and C10) that bind themselves to the service. Specifically, if the OOM adjustment value of the service process is bigger than the values of some of its client processes, its value will be adjusted to the smallest one of all its client processes, and the thread scheduling group will be set to `SCHED_GROUP_DEFAULT`. Thus, the service process will not be killed by RRT, KBP, and KAP.

Implementation: An app process calls `bindService` to bind itself to the service that is running in a separate process. Note that once all client processes disconnect themselves from the service by calling `unbindService`, the service process's priority will not be affected by the client processes.

• Acquiring Published Content Provider (ACP).

Design: A content provider can run in a separate app process (e.g., provider process), and its priority will be affected by its client processes (i.e., C04, C07, and C11). The OOM adjustment and the thread scheduling group of the provider process will be changed following the same rules as the service process mentioned in BRS. Thus, the provider process will not be killed by RRT, KBP, and KAP.

Implementation: To connect itself to the provider process, the app process calls the `acquireContentProviderClient` API or

Table 3: Diehard methods for waking up apps.

Methods	Support Instant App	Method for Killing App Process					
		RRT	FSA	KBP	KAP	KSP	LMK
CSS	✓	●	⊖	●	●	●	●
MSB	✗	●	⊖	●	●	●	●
LAS	✓	●	⊖	●	●	●	●
UJS	✗	●	⊗	●	●	●	●
MAB	✓	●	●	●	●	●	●

1 Meanings of ●, ⊖, and ⊗ are the same as those in Table 2.

the `acquireUnstableContentProviderClient` API, which are declared in the `ContentResolver` class. Note that the priority of the provider process will no longer be affected by its client processes if they are disconnected by calling the APIs, `close` or `release`, which are provided by the `ContentProviderClient` class.

5 WAKING UP APPS

Since none of the methods in Table 2 can prevent an app from being killed by KSP, solely keeping apps alive alone is not enough. Hence, diehard apps also use other methods to wake themselves up after being stopped. Note that, if a component of the stopped app has been launched, the app is waken up.

We propose 5 methods listed in Table 3 to wake up diehard apps. These methods are divided into 3 categories for different scenarios. First, the app utilizes its own service (e.g., a sticky service) to wake itself up (in §5.1). Since this method cannot relaunch normal apps stopped by FSA, we further propose methods in other categories. Second, the app leverages system components to wake itself up. For example, the app can use Alarm service to periodically launch its component to wake itself up. Note that Android system sets several constraints for apps to use the system components. For example, some of the system components are not available for instant apps and cannot timely wake up the app. Hence, apps may adopt the method in the third category. Third, the app relies on other apps to wake it up. That is, the stopped app waits for the wake-up signal (e.g., broadcasts) from other apps. Once received, the signal launches the stopped app's component that responds to the signal, and then the app is waken up.

5.1 Utilizing App Component

A stopped app can wake itself up through its specific components.

• Constructing Sticky Service (CSS).

Design: The unique feature of the sticky service makes it suitable for executing long-time tasks [10]. Specifically, if the process running a sticky service is killed after the service has been started (i.e., its `onStartCommand` callback has been executed), Android will relaunch the service. Thus, diehard apps, which are stopped by RRT, KBP, KAP, KSP, or LMK, can use the sticky service to wake themselves up. However, this method cannot wake up the app stopped by FSA, because FSA makes Android ignore the app's sticky services. *Implementation:* To turn a started service to a sticky one, the service's `onStartCommand` callback should return one of the following constants: `START_STICKY_COMPATIBILITY`, `START_STICKY`, or `START_REDELIVER_INTENT`. Using the former 2 constants, the service can be relaunched by the framework multiple times whereas the service can be relaunched only once if the last constant is

Table 4: Partial System broadcasts for implementing MSB.

Action Name	System Event
ACTION_BOOT_COMPLETED	The device has finished booting.
ACTION_LOCALE_CHANGED	The device's locale has changed.
ACTION_MEDIA_MOUNTED	An external media has been mounted.
ACTION_NEW_OUTGOING_CALL	An outgoing call is about to be placed.
ACTION_TIMEZONE_CHANGED	The timezone has changed.

used. To ensure the `onStartCommand` callback will be invoked by the framework, the app should start the sticky service using the `startService` API. Note that the relaunched service is a background service by default, which will be stopped by AMS if it has been running in the background for more than a minute [1]. To tackle this issue, the diehard app can use our method HFS in §4.2 to bring the relaunched service to the foreground.

5.2 Leveraging System Functionality

Diehard apps can be waken up by exploiting particular system functionality (e.g., system services and broadcasts). We elaborate 3 diehard methods based on app-accessible system functionality.

• Monitoring System Broadcast (MSB).

Design: An app can use broadcast receivers to monitor system events [36]. Once a concerned event happens, the receiver will be launched. Thus, the app, which is stopped by RRT, KBP, KAP, KSP, or LMK, can use this method to wake itself up. However, since FSA makes the stopped app only receive the broadcasts that carry the flag, `FLAG_INCLUDE_STOPPED_PACKAGES`, which is not included in the system broadcasts that can be received by the app [6], MSB cannot wake up normal apps stopped by FSA.

Implementation: The broadcast receiver can be declared in the app's manifest file or dynamically registered by code through the framework API `registerReceiver`. Note that, if the app has been stopped, only the receivers declared in its manifest file can be launched, because Android will only inspect the app's manifest file to determine whether its receivers can respond to the system broadcasts [2]. Thus, a few system broadcasts (e.g., `ACTION_SCREEN_ON`), which can only be received by dynamically registered receivers, cannot wake up the app. Moreover, from Android 8.0, statically declared receivers are no longer permitted to receive the implicit broadcasts (e.g., most of system broadcasts), which do not specify the package name of the target receiver [1]. Hence, only the system broadcasts, which are exempted from these constraints [6], can be used to implement MSB, and we list partial of them in Table 4.

• Leveraging Alarm Service (LAS).

Design: `AlarmManager` allows an app to repeatedly launch its component for executing a certain task. Accordingly, the app, which is stopped by RRT, KBP, KAP, KSP, and LMK, can use this function to wake itself up. However, LAS cannot wake up the app stopped by FSA, because FSA removes the task pending in the Alarm service.

Implementation: To repeatedly launch an app component, the app first constructs a `PendingIntent` task, which specifies the app component to be launched. Then, the app calls `setRepeating` or `setInexactRepeating` declared in the `AlarmManager` class to set the time interval for `AlarmManagerService` to schedule the task.

• Using Job Scheduling Service (UJS).

```

1 // The value of the "intervalMillis" field of oldJob is no smaller than 15*60*1000.
2 public void exploit(JobInfo oldJob) {
3     // Step 1: Store the original JobInfo object using a Parcel container.
4     Parcel in = Parcel.obtain();
5     oldJob.writeToParcel(in, PARCELABLE_WRITE_RETURN_VALUE);
6     // Step 2: Revise the "intervalMillis" value to a smaller value.
7     Parcel out = Parcel.obtain();
8     // reviseParcel(*) is a customized method manipulates the buffer in the Parcel.
9     reviseParcel(in, PARCELABLE_WRITE_RETURN_VALUE, out);
10    // Step 3: Create the JobInfo object using the buffer of the Parcel container.
11    JobInfo newJob = JobInfo.CREATOR.createFromParcel(out);
12    jobScheduler.schedule(newJob);
13 }

```

Figure 3: Adjusting the time interval stored in JobInfo.

Design: Since Android 5.0, an app can ask Android system to periodically execute a job by defining it in the app’s JobService component. More specifically, the app submits a JobInfo object to notify JobSchedulerService to launch the JobService component and execute the job. Thus, the app, which is stopped by RRT, KBP, KAP, KSP, or LMK, can use JobSchedulerService to relaunch its JobService component and wake itself up. However, UJS cannot wake up the app stopped by FSA, because FSA cancels the scheduled job accepted by JobSchedulerService. Moreover, since the JobService component of the instant app cannot be accessed by JobSchedulerService via the package name and class name [7], UJS cannot wake up instant apps.

Implementation: Diehard apps first create the JobInfo object by using the JobInfo.Builder class, and then invokes setPeriodic to specify the time interval for periodically performing the job defined in the JobService component. Note that, after Android 8.0, setPeriodic adjusts the minimum time interval (i.e., the value stored in the intervalMillis field of the JobInfo object) to 15 minutes for optimizing the battery usage. To bypass this limit, we discover and exploit the inconsistency of the time interval check conducted by the local JobInfo object and the remote JobSchedulerService instance¹. Specifically, Android only checks the validity of the time interval in the client side when the app calls setPeriodic, rather than in the server side when JobSchedulerService is going to schedule the job according to the time interval stored in the JobInfo object. Figure 3 illustrates a proof-of-concept (PoC) that can be adopted by the diehard app to bypass the time interval constraint.

5.3 Relying on Third-party Apps

A diehard app can be waken up by colluded third-party apps, which can launch the activity, start the service, or acquire the content provider of the diehard app by explicitly specifying the names of these components. Since the relationship between the diehard app and the colluded app can be easily figured out according to the explicitly specified component names, we do not study this method in this paper. Instead, we design an approach for third-party apps to implicitly wake up the diehard app.

• Monitoring App Broadcast (MAB).

Design: An app can use the receiver declared in its manifest file to monitor the broadcast from third-party apps to wake itself up. Unlike system broadcasts, a well-crafted implicit broadcast can wake up the app stopped by FSA. Hence, the apps stopped by any of 6 process-killing methods can use MAB to wake themselves up.

¹In the latest released Android 10.0, such vulnerability has been fixed.

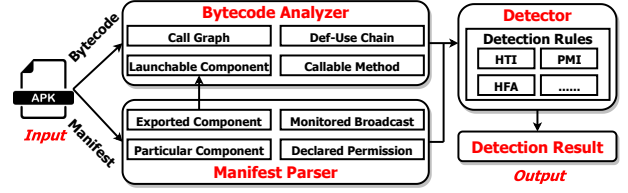


Figure 4: The overview of DiehardDetector.

Implementation: In accordance with the design, the broadcast receiver needs to be statically declared in the manifest file. The implicit app broadcast sent by other apps should satisfy 2 requirements. First, the broadcast should bypass the implicit broadcast constraint since Android 8.0 (i.e., Android no longer permits apps to send implicit broadcasts). Third-party apps can circumvent it by sending broadcast with the flag FLAG_RECEIVER_INCLUDE_BACKGROUND. Second, the app stopped by FSA should be able to receive the implicit broadcast. To achieve it, the third-party apps should let the implicit broadcast carry the flag FLAG_INCLUDE_STOPPED_PACKAGES.

6 DETECTING DIEHARD APPS

We are interested in how prevalent the diehard methods are used by apps in the wild. To conduct a large-scale study on it, we develop a new tool named DiehardDetector to check whether apps have employed the diehard methods described in §4 and §5. After giving an overview of DiehardDetector in §6.1, we detail the modules of DiehardDetector in §6.2 - §6.4.

6.1 Overview

As shown in Figure 4, DiehardDetector consists of 3 modules: manifest parser, bytecode analyzer, and detector. It takes in an app’s APK file and outputs the diehard methods used by this app if any.

• **Manifest Parser** (§6.2): It parses the app’s manifest file to collect the information about exported app components to facilitate the subsequent static analysis on the app’s Dex file. To obtain the necessary information for detecting diehard methods, this module also retrieves the broadcasts monitored by the app, finds the app components with special attributes, and collects the declared permissions.

• **Bytecode Analyzer** (§6.3): It first constructs the call graph and builds the def-use chain by analyzing the app’s Dex files. Then, it finds the app’s components, which cannot be launched, to prune invalid call graph edges caused by the conservative graph construction strategy [12], which assumes that all the app’s components are launchable. Subsequently, it collects the nodes, which denote methods, including app defined methods and framework APIs, in the pruned call graph for identifying diehard methods.

• **Detector** (§6.4): Taking the results of the manifest parser and the bytecode analyzer as inputs, this module discovers the diehard methods used by the app according to the rules defined for the diehard method described in §4 and §5.

6.2 Parsing Manifest File

We collect the following information from the manifest file.

• **Exported App Components.** To collect app components that can be launched externally, DiehardDetector inspects the exported

attribute of each declared app component and stores exported ones to the set $S_{exported}$. Note that, if a component has `intent-filters`, it can receive external intents, and thus DiehardDetector also saves such components to $S_{exported}$.

- **Monitored Broadcasts.** To obtain the broadcasts monitored by the statically declared receivers, DiehardDetector examines the `intent-filters` of each broadcast receiver, and puts the system broadcasts and app broadcasts to 2 sets, namely $S_{syscast}$ and $S_{appcast}$, respectively.

- **Special App Components.** To find the app components that are relevant to diehard methods, DiehardDetector resolves each app component's attributes. Precisely, the service or the content provider with the `process` attribute, which makes the component run in a separate process, will be stored to the set $S_{process}$. Moreover, the activity with the `taskAffinity` attribute, which allows the activity to be placed in a separate activity Task, will be saved to the set $S_{affinity}$. If an activity enables its `excludeFromRecents` attribute, which makes the corresponding activity Task not be associated with a Recent-Task item, such the activity will be recorded to the set $S_{recents}$.

- **Declared Permissions.** To identify the diehard methods (e.g., COW) that need specific permissions, DiehardDetector parses the `uses-permission` tags in the manifest file and stores the declared permissions to the set $S_{permission}$.

6.3 Analyzing Bytecode

To facilitate the detection in §6.4, DiehardDetector performs static analysis to construct an accurate call graph, build def-use chain, and identify launchable components as well as callable methods.

- **Call Graph:** To find the execution path for identifying HFA and HFS, DiehardDetector first uses FlowDroid [12] to build the app's call graph and then prune invalid edges.

Given an app, FlowDroid creates an entry-point and connects it with the callbacks of app components under the assumption that all app components are launchable. That is, the callbacks of app components can always be executed. Then, FlowDroid finds the methods that are reachable from the callbacks to build the call graph. However, since some app components can never be launched, the call graph contains invalid edges, which may bring false positives to the detection results of diehard apps.

To prune invalid edges, DiehardDetector finds launchable app components, keeps the edges that are reachable from the callbacks of launchable app components, and removes the others (i.e., invalid edges). Normally, there are 3 types of launchable app components: (1) exported app components, which can be launched from the external (e.g., users or other apps); (2) the app components, which can be launched by exported app components; (3) the app components, which can be launched by launchable app components. Accordingly, since exported app component are always launchable, DiehardDetector finds the launchable unexported app components to prune invalid edges.

The process for pruning invalid edges consists of 3 steps. In the first step, DiehardDetector finds the unexported app components that can be launched by other app components. In detail, DiehardDetector locates the framework APIs (e.g., those listed in Table 6) for launching the app components. Then, DiehardDetector performs

def-use analysis on the `Intent` objects past to these APIs because the uses of these `Intent` objects contain the names of the launchers (i.e., the app components that launch the unexported app components) and the unexported app components to be launched. In the second step, to ensure the launched unexported app components are launchable app components, DiehardDetector checks whether they are launched by launchable app components. More specifically, DiehardDetector inspects whether the launchers of the unexported app components are launchable app components. If so, such the unexported app components are launchable app components. In the third step, to prune invalid edges, DiehardDetector traverses the call graph from the callbacks of identified launchable app components, and then removes the edges that cannot be accessed.

- **Def-Use Chain:** To find the variable definitions and uses for detecting COW, BRS, and ACP, DiehardDetector builds def-use chains [18] of selected variables. More specifically, to get the window type of the created overlay for detecting COW, DiehardDetector builds def-use chains of the variables that will be assigned to the a parameter of `Window`'s `setAttributes` method or the type parameter of `Window`'s `setType` method. To find the app services that will not be unbound for detecting BRS, DiehardDetector builds def-use chains of `ServiceConnection` objects, which will be passed to the `bindService` API or the `unbindService` API. To find the content providers that will not be released or closed for detecting ACP, DiehardDetector builds def-use chains of `ContentProviderClient` objects, which are either the returned objects of `acquireContentProviderClient` or the base objects of invocations to `ContentProviderClient`'s `release/close` method.

Moreover, to find the app components that will be launched using special `Intent` flags, which are essential for identifying PMI, CSS, and MAB, DiehardDetector builds def-use chains of `Intent` objects passed to framework APIs that are used to launch app components. DiehardDetector stores the activities launched with the `FLAG_ACTIVITY_NEW_TASK` flag to the set S_{task} , saves the services started by `startService` to the set $S_{startService}$, and puts the broadcasts sent with flags, `FLAG_RECEIVER_INCLUDE_BACKGROUND` and `FLAG_INCLUDE_STOPPED_PACKAGE` to the set $S_{sndcast}$.

- **Launchable Components:** To find launchable app components for identifying HFA and HFS, DiehardDetector examines the call graph to get the launchable app components' callbacks, whose method signatures contain the names of the app components. DiehardDetector saves the launchable app components to the set $S_{launchable}$.

- **Callable Methods:** To get the callable methods for recognizing HTI, COW, LAS, and UJS, DiehardDetector stores the nodes in the call graph, each of which corresponds to a method that is reachable from callbacks of launchable app components, to the set $S_{callable}$.

6.4 Detecting Diehard Methods

DiehardDetector identifies various diehard methods according to the rules summarized in Table 5. We explain them as follows.

- (1) **HTI:** DiehardDetector examines $S_{exported}$ and $S_{recents}$ to find whether an exported activity's `excludeFromRecents` attribute is set to `true`. If no activity is found, DiehardDetector will inspect $S_{callable}$ to find whether `setExcludeFromRecents` can be called.

Table 5: Core rules for detecting diehard methods.

Behavior	Detection Rule
HTI	$(S_{exported} \cap S_{recents} \neq \emptyset) \vee ((\text{AppTask.setExcludeFromRecents} \in S_{callable}) \wedge (args[0] = true))$
PMI	$(S_{affinity} \cap S_{task}) \neq \emptyset$
HFA	$\exists \text{ receiver}, (\text{receiver} \in S_{launchable}) \wedge ((\text{path}(\text{onReceive}, \text{*startActivity}) \neq \emptyset) \vee (\text{path}(\text{onReceive}, \text{PendingIntent.getActivity}) \neq \emptyset))$
HFS	$\exists \text{ service}, (\text{service} \in S_{launchable}) \wedge ((\text{path}(\text{onCreate}, \text{startForeground}) \neq \emptyset) \vee (\text{path}(\text{onBind}, \text{startForeground}) \neq \emptyset) \vee (\text{path}(\text{onStartCommand}, \text{startForeground}) \neq \emptyset))$
COW	$(\text{SYSTEM_ALERT_WINDOW} \in S_{permission}) \wedge (((\text{Window.setAttributes} \in S_{callable}) \wedge (args[0].type \geq \text{TYPE_PHONE})) \vee ((\text{Window.setType} \in S_{callable}) \wedge (args[0] \geq \text{TYPE_PHONE})))$
BRS	$\exists \text{ serviceConnection}, (\text{serviceConnection} \in S_{bind}) \wedge (\text{serviceConnection} \notin S_{unbind})$
ACP	$\exists \text{ contentProviderClient}, (\text{contentProviderClient} \in S_{acquire}) \wedge (\text{contentProviderClient} \notin (S_{release} \cup S_{close}))$
CSS	$\exists \text{ service}, ((\text{service} \in S_{exported}) \vee (\text{service} \in S_{startService})) \wedge (\text{service} \in S_{sticky})$
MSB	$\exists \text{ broadcast}, (\text{broadcast} \in \text{system broadcasts that are exempted from the implicit broadcast constraint}) \wedge (\text{broadcast} \in S_{syscast})$
LAS	$(\text{AlarmManager.setInexactRepeating} \in S_{callable}) \vee (\text{AlarmManager.setRepeating} \in S_{callable})$
UJS	$\text{JobInfo\$Builder.setPeriodic} \in S_{callable}$
MAB	$((S_{appcast} \cap S_{sndcast}) \neq \emptyset) \wedge (S_{sndcast} \text{ comes from another app})$

1 The syntax $args[index]$ returns the specific argument of the callable method; The syntax $path(src, tgt)$ obtains the invocation chains that start from the src method and end with the tgt method.

Table 6: Partial APIs for launching app components.

Component	Framework APIs
Activity	Context.startActivity(Intent) Activity.startActivity(Intent) PendingIntent.getActivity(Context, int, Intent, int)
Service	Context.startService(Intent) PendingIntent.getService(Context, int, Intent, int) Context.bindService(Intent, ServiceConnection, int)
Broadcast Receiver	Context.sendBroadcast(Intent) PendingIntent.getBroadcast(Context, int, Intent, int) Context.registerReceiver(BroadcastReceiver, IntentFilter)

If any one of conditions is satisfied, which implies the app will hide its Recent-Task item, HTI is detected.

(2) **PMI**: If an activity is included in both $S_{affinity}$ and S_{task} , PMI is detected, because launching the activity will let the app have multiple activity Tasks, corresponding to multiple Recent-Task items.

(3) **HFA**: DiehardDetector analyzes the call graph to decide whether the `onReceive` callback of a launchable broadcast receiver will call `PendingIntent's startActivity` or `getActivity` to launch the activity. If found, HFA is recognized, because a foreground activity can be launched when the app is running in the background.

(4) **HFS**: DiehardDetector detects HFS by examining callbacks (i.e., `onCreate`, `onBind`, and `onStartCommand`) of each app service in $S_{launchable}$ to determine whether `startForeground` will be invoked by the callbacks. If so, it denotes that the app will launch the foreground service, and thus HFS is detected.

(5) **COW**: DiehardDetector first checks $S_{permission}$ to find out whether the `SYSTEM_ALERT_WINDOW` permission has been requested by the app. If so, DiehardDetector analyzes the def-use chains of variables passed to the `Window's setAttributes` or `setType` method to get the type of the window created by the app. If the window type's value is no smaller than that of `TYPE_PHONE`, COW is found because the app will create an overlay [32].

(6) **BRS**: The core for detecting BRS is to find the remote app service that can be connected through `bindService`, and such connection will never be broke via `unbindService`. Accordingly, DiehardDetector analyzes the def-chains of `ServiceConnection` objects passed to `bindService` or `unbindService`, and stores the found variable definitions to sets, S_{bind} and S_{unbind} , respectively. If the intersection of these two sets is not a null set, which implies some service bindings will never be broken, DiehardDetector then analyzes the

def-use chains of `Intent` objects passed to `bindService` to find the bound service. If the service is also included in $S_{process}$, the remote service will use BRS to escalate its process priority.

(7) **ACP**: DiehardDetector identifies ACP by finding the acquired `ContentProviderClient` objects that will not be released or closed. We store the definitions of the acquired objects (i.e., the returned objects of either `acquireUnstableContentProviderClient` or `acquireContentProviderClient`), the closed objects (i.e., the base objects of the invocations to `ContentProviderClient's close` method), and the released objects (i.e., the base objects of the invocations to `ContentProviderClient's release` method), to the sets $S_{acquire}$, S_{close} , and $S_{release}$, respectively. If there is an object, which is included in $S_{acquire}$ but is excluded from $S_{release}$ and S_{close} , and its corresponding content provider is included in $S_{process}$, we find a remote content provider that will use ACP to elevate its process priority.

(8) **CSS**: DiehardDetector examines the return value of each app service's `onStartCommand` callback to find sticky services, and stores them to the set S_{sticky} . If the intersection of S_{sticky} and $S_{startService}$ is not a null set, CSS is recognized, because the app will start the sticky service by using `startService`,

(9) **MSB**: If the app declares receivers to monitor the system broadcasts, which are exempted from the implicit broadcast constraint, we consider that such an app implements MSB.

(10) **LAS**: To recognize LAS, DiehardDetector inspects $S_{callable}$ to find out whether `setRepeating` or `setInexactRepeating` declared in the `AlarmManager` class will be called by the app. If so, it suggests the app will use Alarm service to relaunch its component.

(11) **UJS**: DiehardDetector checks $S_{callable}$ to figure out whether the `setPeriodic` method defined in the `JobInfo$Builder` class will be called by the app. If so, the `JobService` component of the app will be relaunched, and thus UJS is found.

(12) **MAB**: If an app broadcast included in $S_{appcast}$ is presented in $S_{sndcast}$ of another app, MAB is detected because the app under examination listens to the well-crafted broadcast from another app,

7 EVALUATION

In this section, we evaluate the applicability of our diehard methods, the performance of DiehardDetector, and the prevalence of diehard apps in the wild by answering the following 3 research questions.

Table 7: Diehard methods found in data-2k apps.

Behavior	#app	#app'	Δapp	Behavior	#app	#app'	Δapp
HTI	118	118	+0	ACP	0	0	+0
PMI	10	13	+3	CSS	72	91	+19
HFA	2	2	+0	MSB	452	452	+0
HFS	139	240	+101	LAS	158	165	+7
COW	11	14	+3	UJS	19	19	+0
BRS	0	0	+0	MAB	0	0	+0

RQ1: Do our diehard methods work on most popular Android versions, ranging from Android 5.1 to Android 10.0?

RQ2: How is the performance of DiehardDetector?

RQ3: How prevalent are diehard apps?

Data Set: We use 3 data sets including data-3, data-2k, and data-80k for answering the 3 research questions, respectively. To answer

RQ1, we developed a normal app and an instant app, both of which implement all available diehard methods for their app types. We also developed an auxiliary normal app for verifying MAB. To answer **RQ2**, we downloaded 2,080 open-source apps from F-Droid [4] to form the benchmark. To answer **RQ3**, we randomly crawled 81,237 apps from Google Play [5].

7.1 RQ1: Applicability of Diehard Methods.

Methodology: To assess the applicability of our diehard methods, we run each app in data-3 on various versions of official Android systems, which are released by Google.

Result: All our diehard methods can prolong the liveness time of apps running on the official Android systems, ranging from 5.1 to 10.0. Since Android 10.0 has patched the vulnerability in the time interval check (mentioned in §5.2), we cannot change the minimum time interval (15 mins) for `JobSchedulerService` to execute the periodic job and thus UJS cannot timely relaunch the app.

Answer to RQ1: All our 12 diehard methods are applicable to popular Android systems, ranging from Android 5.1 to Android 10.0. However, Android 10.0 constraints the ability of UJS.

Table 8: Apps using diehard methods in data-80k.

Behavior	#app	Ratio	Behavior	#app	Ratio
HTI	1,878	10.79%	ACP	0	0.00%
PMI	424	2.44%	CSS	1,933	11.14%
HFA	125	0.72%	MSB	12,413	71.30%
HFS	2,243	12.88%	LAS	6,452	37.06%
COW	264	1.52%	UJS	130	0.75%
BRS	21	0.12%	MAB	0	0.00%

Table 9: Number of diehard methods used in each app.

data-2k (632)			data-80k (17,410)		
Number	#app	Ratio	Number	#app	Ratio
1	363	57.44%	1	11,213	64.41%
2	205	32.44%	2	4,444	25.53%
3	50	7.91%	3	1,375	7.90%
4	12	1.90%	4	272	1.56%
5	2	0.31%	5	75	0.43%
6	0	0.00%	6	31	0.17%

7.2 RQ2: Performance of DiehardDetector.

Methodology: To evaluate the performance of DiehardDetector, we first use it to analyze the apps in data-2k, and then manually analyze the source code of the apps to determine whether they do (or do not) have diehard behaviors. More specifically, we manually analyzed 500 randomly selected apps in data-2k, 250 of which are detected diehard apps while the remaining 250 are normal apps.

Result: Table 7 lists the detection results, where *#app* denotes the number of apps having a specific diehard behavior. Since FlowDroid fails to process 9 apps in data-2k due to the over-long analysis time (> 30 mins), DiehardDetector successfully analyzed 2,071 apps, and found 632 (30.52%) of them use diehard methods.

After analyzing 250 detected diehard apps manually, we did not find any false positives. For the other 250 apps, we found 14 false negatives, which make FlowDroid throw an exception when constructing the call graph. Consequently, their call graphs include neither nodes nor edges so that DiehardDetector cannot detect their diehard methods (except for PMI) and thus treats them as non-diehard apps. Accordingly, the precision and the recall of DiehardDetector is 100% and 94.70%, respectively.

To evaluate the effect of pruning the call graph (§6.3), we compare the detection results (*#app*) with those (*#app'*) achieved without removing invalid call graph edges. Specifically, Δapp shows the increased number of identified apps having a specific diehard behavior (i.e., $\Delta app = \#app' - \#app$). In detail, if invalid edges are not removed, DiehardDetector will cause 53 false positives. Moreover, we notice that over 40% of *#app'* for HFS are false positives because the recognized foreground services will never be launched. Hence, removing invalid call graph edges is important to DiehardDetector.

Answer to RQ2: DiehardDetector can identify diehard apps with high precision (100%) and recall (94.7%).

Table 10: Ratio of diehard apps in various app categories.

Category	Ratio	Category	Ratio	Category	Ratio
Personalization	40.26%	Social	25.26%	Entertainment	18.26%
Communication	39.44%	Video Players	24.91%	Photography	18.04%
News&Magazines	31.80%	Lifestyle	22.33%	Finance	17.31%
Tools	31.47%	Health&Fitness	22.17%	Education	14.39%
Music&Audio	29.40%	Business	21.87%	Book&Reference	13.92%
Shopping	27.60%	Travel&Local	20.02%	Games	10.10%

7.3 RQ3: Prevalence of Diehard Apps.

Methodology: To detect diehard apps in the wild and investigate the popularity of diehard methods, we apply DiehardDetector to analyzing the apps in data-80k crawled from Google Play.

Result: The average time for DiehardDetector to analyze an app is 45 seconds. Table 8 lists the overall result, where *Ratio* is the ratio of diehard apps with a specific diehard method. It shows that 17,410 (around 21%) of Google Play apps in our dataset use diehard methods, and thus diehard apps are prevalent in the app ecosystem. Moreover, MSB, LAS, CSS, HFS, and HTI are the most popular diehard methods, and roughly 70% of diehard apps implement MSB. It is interesting that ACP and MAB were not found in data-2k and data-80k. Thus, we discover 2 new ways for apps to keep their processes alive and wake themselves up.

Since a diehard app may adopt more than one diehard method, we also analyze the number of diehard methods used in each diehard app, and the results are shown in Table 9. In detail, more than 35% of diehard apps adopt more than one diehard methods, but very few diehard apps employ more than four diehard methods.

We also study the distribution of diehard apps in different app categories, and the results are listed in Table 10. It shows that more than 40% of apps in Personalization category use diehard methods. Moreover, the categories including Communication, News & Magazines, and Tools have more than 30% diehard apps.

Answer to RQ3: Around 21% of Google Play apps in our dataset adopt diehard methods. The top 5 ones are MSB, LAS, CSS, HFS, and HTL. ACP and MAB have not been widely used yet. Moreover, diehard methods are widely used by apps in categories like Personalization, Communication, News & Magazines, and Tools.

8 THREAT TO VALIDITY

The threats to the external validity of DiehardDetector come from 2 aspects. First, some special app samples may negatively affect the performance of DiehardDetector. More precisely, due to the intrinsic problem of static analysis, DiehardDetector cannot handle the apps that adopt advanced obfuscations or packing techniques to prevent the app's bytecode from being analyzed. To tackle this issue, we may use deobfuscators (e.g., TIRO [29]) and unpackers (e.g., PackerGrind [30, 31]) to recover the protected bytecode. Moreover, some apps may use native code to call the methods (e.g., `startActivity`) relevant to constructing the app's call graph and detecting diehard methods. FlowDroid may miss them because it cannot analyze the native code of the app. To mitigate this problem, we may employ JN-SAF [28] to get the method invocations implemented in the app's native code. Second, although we design 12 diehard methods, we might miss some other diehard methods adopted by apps in the wild. Nevertheless, this is the most comprehensive study of diehard behaviors, and over half of the proposed diehard methods have never been mentioned by existing studies.

9 DISCUSSION

• **Applicability of Diehard Methods:** In section §7.1, we evaluate the applicability of diehard methods using different versions of the Android systems released by Google. However, to achieve the purpose of prolonging the battery life, mobile vendors will introduce their own process-killing methods to their customized Android systems, which may compromise the effectiveness of the presented diehard methods. In future work, we will further evaluate the applicability and the effectiveness of the proposed diehard methods using different mobile vendors' customized Android systems.

• **Intentional Diehard Behaviors:** In this paper, we do not distinguish the diehard behavior that is intentionally implemented by the adversary and the one that is carelessly produced by the benign developer. In future work, we intend to differentiate these two types of diehard behaviors. For instance, if an app relaunches a dead app process after it checks the aliveness state of this process, we may consider such procedure implies that the diehard behavior is intentionally performed. Additionally, if the behavior for keeping the app process alive does not involve any user interactions, we may also treat it as an intentionally performed diehard behavior.

10 RELATED WORK

This section introduces the studies that are most related to finding diehard apps. First, recent studies identify unusual app behaviors. Shan et al. [24] propose a static analysis tool to detect self-hiding behaviors, which prevent the app from being noticed by users. OverlayChecker [32] runs the app in an emulator to examine whether an overlay window will be launched by the app at runtime. DDAX [25] performs static analysis to find the app that abuses the device administrator, a sensitive functionality provided by Android system. Shao et al. [26] developed a tool to analyze an app's lifecycle, which may be used to detect some diehard methods, e.g., HFS, CSS, MSB, LAS, and UJS. However, it cannot detect other diehard methods proposed by us. Moreover, we are the first to reveal the limitations of existing process-killing methods, and exploit them to design diehard methods. It is worth noting that 7 of 12 proposed diehard methods were not mentioned in [26].

Second, researchers studied the factors affecting the energy consumption of Android apps. EnergyPatch [14] and μ Droid [19] pointed out that improper operations on energy-intensive hardware components cause apps to exhibit poor energy consumption. HOT-PEPPER [15] reveals that the bad implementation practices in apps have negative impacts on their energy consumption. Oliveira et al. [21] and Chowdhury et al. [16] found that the programming languages used to develop apps and the runtime execution logs affect their energy consumption. GEMMA [20] optimizes the colors used by apps to reduce the energy consumption on smartphones' displays. However, none of them mentioned that diehard behaviors can affect the energy consumption of apps.

11 CONCLUSION

We conduct the first systematic investigation on diehard apps and diehard methods. By revealing and exploiting the limitations of existing process-killing methods, we design 12 practical diehard methods for keeping alive app processes or waking up the stopped app, which work on popular Android versions from 5.1 - 10.0. To automatically identify the presence of diehard methods, we develop DiehardDetector that can accurately and quickly detect our diehard methods. The extensive experimental results show that DiehardDetector achieves high precision and recall. By applying DiehardDetector to more than 80k apps fetched from Google Play, we observe that around 21% of apps have adopted diehard methods, suggesting that diehard apps are prevalent in the wild. To help users understand the diehard behaviors, app stores may ask developers to describe such behaviors in their app's privacy policies[34, 35] and check whether the apps have disclosed all their diehard behaviors[33].

12 ACKNOWLEDGEMENT

We thank the anonymous reviewers for their helpful comments. This research is partially supported by the Hong Kong RGC Projects (No. 152279/16E, 152223/17E, CityU C1008-16G) and the National Natural Science Foundation of China (No. 61702045, No. 61872438) and Leading Innovative and Entrepreneur Team Introduction Program of Zhejiang (2018R01005) and Zhejiang Key R&D (2019C03133) and the National Science Foundation under Grant (No. 1953893, 1953813, and 1951729).

REFERENCES

- [1] 2020. Background Execution Limits. <https://developer.android.com/about/versions/oreo/background>.
- [2] 2020. Broadcasts Overview. <https://developer.android.com/guide/components/broadcasts>.
- [3] 2020. Don't kill my app! <https://dontkillmyapp.com/>.
- [4] 2020. F-Droid. <https://f-droid.org>.
- [5] 2020. Google Play Store. <https://play.google.com/store>.
- [6] 2020. Implicit Broadcast Exceptions. <https://developer.android.com/guide/components/broadcast-exceptions>.
- [7] 2020. Launch Instant App. <https://developers.google.com/android/reference/com/google/android/gms/instantapps/Launcher>.
- [8] 2020. Recents Screen. <https://developer.android.com/guide/components/activities/recents>.
- [9] 2020. Smartphone users still want long-lasting batteries more than shatterproof screens. <https://today.yougov.com/topics/technology/articles-reports/2018/02/20/smartphone-users-still-want-longer-battery-life>.
- [10] 2020. Sticky Service. https://developer.android.com/reference/android/app/Service#START_STICKY.
- [11] 2020. Understand Tasks and Back Stack. <https://developer.android.com/guide/components/activities/tasks-and-back-stack>.
- [12] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Octeau, and Patrick McDaniel. 2014. Flow-Droid: Precise Context, Flow, Field, Object-sensitive and Lifecycle-aware Taint Analysis for Android Apps. In *Proc. PLDI*.
- [13] Kathy Wain Yee Au, Yi Fan Zhou, Zhen Huang, and David Lie. 2012. PScout: Analyzing the Android Permission Specification. In *Proc. CCS*.
- [14] Abhijeet Banerjee, Lee Kee Chong, Clément Ballabriga, and Abhik Roychoudhury. 2018. EnergyPatch: Repairing Resource Leaks to Improve Energy-Efficiency of Android Apps. *IEEE Transactions on Software Engineering* (2018).
- [15] Antonin Carette, Mehdi Adel Ait Younes, Geoffrey Hecht, Naouel Moha, and Romain Rouvoy. 2017. Investigating the energy impact of Android smells. In *Proc. SANER*.
- [16] Shaiful Chowdhury, Silvia Di Nardo, Abram Hindle, and Zhen Ming Jiang. 2018. An exploratory study on assessing the energy impact of logging on Android applications. *Empirical Software Engineering* (2018).
- [17] Yanick Fratantonio, Chenxiong Qian, Simon P Chung, and Wenke Lee. 2017. Cloak and Dagger: From Two Permissions to Complete Control of the UI Feedback Loop. In *Proc. S&P*.
- [18] Mary Jean Harrold and Mary Lou Soffa. 1994. Efficient Computation of Interprocedural Definition-Use Chains. *ACM Trans. Program. Lang. Syst.* (1994).
- [19] Reyhaneh Jabbarvand and Sam Malek. 2017. μ Droid: An Energy-Aware Mutation Testing Framework for Android. In *Proc. FSE*.
- [20] Mario Linares-Vásquez, Carlos Bernal-Cárdenas, Gabriele Bavota, Rocco Oliveto, Massimiliano Di Penta, and Denys Poshyvanyk. 2017. GEMMA: Multi-objective Optimization of Energy Consumption of GUIs in Android Apps. In *Proc. ICSE*.
- [21] Wellington Oliveira, Renato Oliveira, and Fernando Castor. 2017. A Study on the Energy Consumption of Android App Development Approaches. In *Proc. MSR*.
- [22] Chuangang Ren, Peng Liu, and Sencun Zhu. 2017. WindowGuard: Systematic Protection of GUI Security in Android. In *Proc. NDSS*.
- [23] Chuangang Ren, Yulong Zhang, Hui Xue, Tao Wei, and Peng Liu. 2015. Towards Discovering and Understanding Task Hijacking in Android. In *Proc. USENIX Security*.
- [24] Zhiyong Shan, Iulian Neamtii, and Raina Samuel. 2018. Self-hiding Behavior in Android Apps: Detection and Characterization. In *Proc. ICSE*.
- [25] Zhiyong Shan, Raina Samuel, and Iulian Neamtii. 2019. Device Administrator Use and Abuse in Android: Detection and Characterization. In *Proc. MobiCom*.
- [26] Yuru Shao, Ruowen Wang, Xun Chen, Ahemd M. Azab, and Z. Morley Mao. 2019. A Lightweight Framework for Fine-Grained Lifecycle Control of Android Applications. In *Proc. EuroSys*.
- [27] Yulei Sui, Ding Ye, and Jingling Xue. 2014. Detecting memory leaks statically with full-sparse value-flow analysis. *IEEE Transactions on Software Engineering* (2014).
- [28] Fengguo Wei, Xingwei Lin, Xinming Ou, Ting Chen, and Xiaosong Zhang. 2018. JN-SAF: Precise and Efficient NDK/JNI-Aware Inter-Language Static Analysis Framework for Security Vetting of Android Applications with Native Code. In *Proc. CCS*.
- [29] Michelle Y Wong and David Lie. 2018. Tackling runtime-based obfuscation in Android with TIRO. In *Proc. USENIX Security*.
- [30] Lei Xue, Xiapu Luo, Le Yu, Shuai Wang, and Dinghao Wu. 2017. Adaptive unpacking of Android apps. In *Proc. ICSE*.
- [31] L. Xue, H. Zhou, X. Luo, L. Yu, D. Wu, Y. Zhou, and X. Ma. 2020. PackerGrind: An Adaptive Unpacking System for Android Apps. *IEEE Transactions on Software Engineering* (2020).
- [32] Yuxuan Yan, Zhenhua Li, Qi Alfred Chen, Christo Wilson, Tianyin Xu, Ennan Zhai, Yong Li, and Yunhao Liu. 2019. Understanding and Detecting Overlay-based Android Malware at Market Scales. In *Proc. MobiSys*.
- [33] L. Yu, X. Luo, J. Chen, H. Zhou, T. Zhang, H. Chang, and H. Leung. 2019. PPChecker: Towards Accessing the Trustworthiness of Android Apps' Privacy Policies. *IEEE Transactions on Software Engineering* (2019).
- [34] Le Yu, Tao Zhang, Xiapu Luo, and Lei Xue. 2015. AutoPPG: Towards Automatic Generation of Privacy Policy for Android Applications. In *Proc. SPSM*.
- [35] L. Yu, T. Zhang, X. Luo, L. Xue, and H. Chang. 2017. Towards Automatically Generating Privacy Policy for Android Apps. *IEEE Transactions on Information Forensics and Security* (2017).
- [36] Yajin Zhou and Xuxian Jiang. 2012. Dissecting Android Malware: Characterization and Evolution. In *Proc. S&P*.